A Project Report

On

SMARTHEALTH AI PLATFORM – AI-ASSISTED TELEMEDICINE WEB APPLICATION

Submitted by

PARAS PRAVIN BARASKAR

Seat No.510515831

ANKIT GUPTA

Seat No.51051530

VINAY RAJESH GAUD

Seat No.510515835

In partial fulfillment for the award of the degree

of

BACHELOR OF SCIENCE

In

COMPUTER SCIENCE

Under the guidance

Of

PROF. T.Sujah



THEEM COLLEGE OF ARTS, COMMERCE AND SCIENCE

(Affiliated to University of Mumbai)

BOISAR, MAHARASHTRA – 401501

Sem V – 2026-27

THEEM COLLEGE OF ARTS, COMMERCE & SCIENCE

(Affiliated to University of Mumbai)

BOISAR-401501

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that **Paras Pravin Baraskar -Ankit Gupta -Vinay Gaud** , **seat no.510515831-510515830-510515835**

TY.BSC.(CS)

has satisfactorily completed the Project **SmartHealth AI Platform – AI-Assisted Telemedicine Web Application**, to be submitted in the partial fulfillment for the award of **Bachelor of Science in Computer Science** during the academic year **2026 – 2027.**

Internal Guide **Coordinator**

External Examiner **HOD**

Date: **College Seal**

DECLARATION

We, Mr. **paras pravin baraskar**, **Ankit Gupta ,Vinay Gaud** student's of **Theem College of Arts, Commerce** and Science studying in **bachelor of science in computer science** program, hereby declare that I have completed the project titled **smarthealth ai platform – ai-assisted telemedicine web application** during the academic year 2026 – 2027.

This project has been developed under the guidance of my project guide. The work presented in this project report is based on my own efforts and understanding. The information and resources used during the development of this application have been properly acknowledged in the references section.

I further declare that this project has not been submitted to any other university or institution for the award of any degree or diploma.

Place: Boisar

Date: _____

Student Name: **Paras Pravin Baraskar**

Seat No. **510515831**

Student Name: **Ankit Gupta**

Seat No. **510515830**

Student Name: **Vinay Rajesh Gaud**

Seat No. **510515835**

Class: **T.Y.B.Sc. Computer Science**

Signature: _____

Signature: _____

Signature: _____

ACKNOWLEDGEMENT

I sincerely express my gratitude to my respected Project Guide, **Prof. T.Sujah** for providing valuable guidance, suggestions and continuous support throughout the development of **"SmartHealth AI Platform – AI-Assisted Telemedicine Web Application**."

I am thankful to the Head of the Department and all faculty members for their academic guidance, technical support and encouragement. I also thank Theem College of Arts, Commerce and Science for providing the necessary infrastructure and resources for completing this project.

I am grateful to my classmates and friends for their valuable feedback and cooperation during development and testing. I also thank my parents and family for their constant motivation and support.

Finally, I thank everyone who directly or indirectly contributed to the successful completion of this project. This project has been a valuable learning experience that improved my technical and problem-solving skills.

ABSTRACT

SmartHealth AI Platform – AI-Assisted Telemedicine Web Application is a web-based application designed to help patients get preliminary health guidance, consult a doctor online and keep a record of their consultations in an organized way.

Users can register and sign in, describe their symptoms to a rule-based AI assistant, choose a specialist, join a simulated video consultation, receive a read-only electronic prescription, pay for the medicines through a simulated payment flow and review their consultation history. The system uses Firebase Authentication for secure sign-in and Firebase Realtime Database to store each patient's history under a private, user-specific path.

The project reduces the effort of travelling and waiting for minor consultations and brings triage, consultation, prescription, payment and records into one flow. It also provides practical experience in frontend development, cloud authentication, database design, testing and deployment. It is an academic demonstration: the triage, video call and payment are simulated, and the prescriptions must not be used for real treatment.

In the future, SmartHealth can be enhanced with real video consultation, a doctor portal, model-based triage, a real payment gateway, appointment scheduling and notifications.

TABLE OF CONTENTS

Sr. No.	Section Title / Description	Page No.
—	Cover Page	1
—	Certificate	2
—	Declaration	3
—	Acknowledgement	4
—	Abstract	5
—	Table of Contents	6
—	List of Figures	9
1	**Chapter 1 PROBLEM IDENTIFICATION & FEASIBILITY STUDY**	**10**
1.1	Identification of a Real-World Problem	10
1.2	Problem Justification	10
1.3	Scope Definition	11
1.4	Stakeholder Identification	12
1.5	Feasibility Study	13
1.5.1	Technical Feasibility	13
1.5.2	Economic Feasibility	14
1.5.3	Operational Feasibility	14
2	**Chapter 2 REQUIREMENT ENGINEERING**	**16**
2.1	Functional Requirements Specification	16
2.2	Non-Functional Requirements	17
2.3	Use-Case Analysis	18
2.4	Requirement Prioritization	20
2.5	Constraints and Assumptions	20
3	**Chapter 3 SOFTWARE DEVELOPMENT LIFE CYCLE (SDLC) PLANNING**	**22**
3.1	Selection of SDLC Model	22
3.2	Work Breakdown Structure (WBS)	23
3.3	Project Timeline	23
3.4	Gantt Chart	24
3.5	Resource Planning	24
4	**Chapter 4 SYSTEM MODELING USING UML**	**26**
4.1	Use Case Diagram	26
4.2	Class Diagram	27
4.3	Sequence Diagram	29
4.4	Activity Diagram	31
4.5	Entity Relationship (ER) Diagram	33
4.6	Deployment Diagram	34
5	**Chapter 5 SYSTEM ARCHITECTURE DESIGN**	**36**
5.1	Frontend Architecture	36

Sr. No.	Section Title / Description	Page No.
5.2	Backend Architecture	38
5.3	Database Schema Design	39
5.4	API Structure	40
5.5	Security Considerations	41
6	**Chapter 6 CODING OF WEBSITE**	**43**
6.1	Frontend coding	43
6.1.1	index.html	43
6.1.2	style.css	44
6.1.3	script.js	44
6.2	Backend coding	47
6.2.1	firebase.js	47
6.2.2	Database access functions (script.js)	48
6.2.3	Database security rules	48
6.2.4	firebase.json	48
7	**Chapter 7 APPLICATION DEVELOPMENT**	**49**
7.1	Frontend Implementation	49
7.2	Backend Implementation	51
7.3	Database Integration	52
7.4	Authentication and Validation	53
7.5	Error Handling	54
8	**Chapter 8 INTEGRATION & SYSTEM TESTING**	**55**
8.1	Unit Testing	55
8.2	Black-Box Testing	56
8.3	Integration Testing	56
8.4	Test Case Preparation	57
8.5	Bug Tracking	59
9	**Chapter 9 APPLICATION DEPLOYMENT**	**61**
9.1	Cloud Deployment	61
9.2	Local Hosting	61
9.3	APK Build	61
9.4	Server Configuration	62
9.5	Version Control Using GitHub	62
10	**Chapter 10 PERFORMANCE & SECURITY TESTING**	**64**
10.1	Basic Load Testing	64
10.2	Input Validation Checks	65
10.3	Security Validation	66
11	**Chapter 11 FINAL DOCUMENTATION**	**68**
11.1	Technical Report	68
11.2	User Manual	71
11.3	Application Screenshots	72

Sr. No.	Section Title / Description	Page No.
11.4	Source Code Documentation	74
12	**Chapter 12 REFERENCE**	**78**
13	**Chapter 13 GLOSSARY**	**79**
14	**Chapter 14 APPENDICES – SMARTHEALTH**	**81**
	Appendix A – Survey Questionnaire	81
	Appendix B – Survey Data Sheet	81
	Appendix C – Sample Test Data	81
	Appendix D – Test Cases	82
	Appendix E – User Feedback	82

LIST OF FIGURES

Figure No.	Caption / Description	Page
Fig. 3.1	Iterative-incremental model used for SmartHealth	22
Fig. 3.2	Work breakdown structure of the SmartHealth project	23
Fig. 3.3	Gantt chart of the 14-week plan	24
Fig. 4.1	Use case diagram of SmartHealth	26
Fig. 4.2	Class diagram (logical classes)	28
Fig. 4.3	Sequence diagram: sign-in	29
Fig. 4.4	Sequence diagram: consultation and prescription	30
Fig. 4.5	Sequence diagram: medicine payment	31
Fig. 4.6	Activity diagram: consultation, prescription and payment	32
Fig. 4.7	Activity diagram: authentication and lockout	33
Fig. 4.8	Logical ER diagram	34
Fig. 4.9	Deployment diagram	35
Fig. 5.1	Layers of the frontend	36
Fig. 5.2	Two-tier architecture. The ES module script exposes Firebase calls on window so the classic script can reach them.	37
Fig. 5.3	Context-level data flow diagram	38
Fig. 5.4	Website navigation of SmartHealth	38
Fig. 5.5	Backend architecture	38
Fig. 6.1	Frontend structure of SmartHealth	43
Fig. 6.2	Backend structure (Firebase services)	47
Fig. 9.1	Git and GitHub workflow	62

SCREENS OF THE APPLICATION (SCHEMATIC WIREFRAMES)

Figure No.	Caption / Description	Page
Fig. 11.1	Screens: login / sign-up (left) and home (right) - schematic wireframes	73
Fig. 11.2	Screens: AI assistant (left) and telemedicine (right) - schematic wireframes	73
Fig. 11.3	Screens: my prescriptions (left) and my history (right) - schematic wireframes	74
Fig. 11.4	Screen: payment modal - schematic wireframe	74

1. Problem Identification & Feasibility Study

This section identifies the real-world problem addressed by SmartHealth, justifies why a software solution is appropriate, draws the boundary of the project, names the people affected by it and finally tests whether the project is technically, economically and operationally feasible.

1.1 Identification of a Real-World Problem

Access to a qualified doctor is constrained by three practical barriers: **distance**, **waiting time** and **paperwork**. A patient in a remote village may travel several hours for a ten-minute appointment. A patient in a city may wait days for a slot. In both cases the prescription that results from the visit is a paper slip that is easily lost, hard to read and impossible to verify afterwards.

At the same time, a large share of first contacts with the healthcare system are low-severity complaints - a cold, a headache, a mild fever - where the real value of the visit is reassurance and a decision about whether escalation is needed. Handling that first decision digitally frees clinical time for the cases that genuinely require it, and gets the patient to the right kind of specialist sooner.

Existing digital options address only part of this chain. Clinic management software serves staff, not patients. Symptom checkers return a list of possible conditions and then stop. Commercial teleconsultation platforms solve distance, but are closed systems whose design cannot be studied or reused for learning. The problem selected for this project is therefore not the absence of any digital tool, but the absence of a simple, **integrated** flow from "I do not feel well" to "I have seen a specialist, hold a prescription and have a record of it".

Table 1.1: Observed pain points in the conventional consultation process

Pain point	Effect on the patient	Effect on the health system
Physical distance to the clinic	Travel time and cost; consultation postponed	Rural patients reach doctors late, with advanced conditions
Long waiting times	Lost working hours; patient may skip the visit	Crowded outpatient departments
Unnecessary visits for minor complaints	Cost and exposure to infection in waiting rooms	Clinical time spent on cases that need only advice
Paper prescriptions	Lost, illegible or altered slips	No verifiable trail; risk of dispensing errors
Scattered records	Patient cannot show past consultations	Doctor repeats history-taking at every visit
Disconnected tools	Chatbot, call, prescription and pharmacy are separate steps	Information re-entered at each step

1.2 Problem Justification

A software solution is justified because every barrier listed in Table 1.1 is a problem of **information flow** rather than of medicine: where the patient is, when the doctor is free, what was prescribed and where it is stored. These are exactly the problems that networked software handles well. The justification rests on four arguments.

1. **Reach.** A browser-based application is available wherever there is a connection, on a laptop or a phone, without installing an app. This directly removes the distance barrier.
2. **Triage at the front door.** Even a simple, transparent rule-based assistant can separate urgent symptoms (for example chest pain or breathing difficulty) from routine ones and point the patient to the right specialist. A patient who is told clearly to seek urgent care, or told that a routine complaint can be handled by a general physician, makes a better first decision.
3. **A single, unbroken workflow.** The value of the platform lies in the joins between steps. Triage names a specialist who exists on the platform; ending a call with that specialist produces the prescription; the prescription contains the button that pays for the medicines; the payment and the consultation land in one history record. The patient never re-enters context.
4. **Persistent, patient-owned records.** Storing each consultation under the patient's own account in a cloud database means the record survives logout, device change and lost paper.

Academic justification. The project also exercises the core competencies expected at this stage of the degree: requirement engineering, UML modelling, architecture design, client-side programming, use of a cloud backend-as-a-service, authentication and access-control design, testing, deployment and documentation. A telemedicine workflow is rich enough to need all of them while still being small enough to complete as a single-developer project.

Table 1.2: Problem-to-solution mapping

Problem	SmartHealth feature	Module
Distance	Browser-based consultation with a listed specialist	M4 Telemedicine
Waiting time	On-demand consultation; no slot booking needed	M4 Telemedicine
Unnecessary visits	AI assistant gives preliminary guidance 24x7	M3 AI assistant
Paper prescriptions	Generated, read-only, watermarked e-prescription	M5 Prescription
Scattered records	Chronological consultation history stored in the cloud	M7 History
Disconnected tools	Triage -> call -> prescription -> payment -> history in one flow	M3-M7
Unauthorised access	Per-user authentication and UID-scoped data path	M1 Authentication

1.3 Scope Definition

A clear boundary prevents a student project from growing without limit. The scope of SmartHealth is limited to the **patient side** of a teleconsultation service.

Table 1.3: Project scope

In scope	Out of scope
Patient registration and login with email and password	A doctor-side portal or doctor login
Free-text symptom triage by a rule-based assistant, as a full page and as a floating widget	Machine-learning or clinical decision support from a trained model
Listing of specialists with name, specialty, experience and rating	Doctor availability, appointment slots and scheduling
Simulated video consultation session with mic, camera and end-call controls	Real peer-to-peer video transport (WebRTC)
Automatic e-prescription matched to the doctor's specialty	Prescriptions composed by a licensed practitioner
Medicine payment flow with UPI, Google Pay, PhonePe, Paytm, card and net banking (simulated)	Real payment settlement and pharmacy inventory
Cloud-stored consultation history, newest first	Regulatory certification (Telemedicine Practice Guidelines, DISHA, HIPAA)
Responsive layout for desktop and mobile browsers	A separately built native mobile application

Deliverables

- A working four-file web application (`index.html`, `style.css`, `firebase.js`, `script.js`).
- A configured Firebase project with Authentication enabled and Realtime Database security rules published.
- This documentation (design, implementation, test and user documentation).
- A source repository suitable for hosting on GitHub.

1.4 Stakeholder Identification

A stakeholder is any person or organisation that affects, or is affected by, the system. Identifying them early makes sure that requirements are not written from the developer's point of view alone.

Table 1.4: Stakeholder register

Stakeholder	Role	Main interest	Influence
Patient (primary user)	Registers, chats, consults, pays, reviews history	Quick advice, safe handling of data, simple flow	High

Stakeholder	Role	Main interest	Influence
Doctor (future user)	Consults patients; would author prescriptions in a future portal	Accurate case information; control of the prescription	Medium (currently represented by a static roster)
Developer / student	Designs, builds, tests and documents the system	A working, well-documented prototype	High
Project guide	Reviews design and progress	Academic quality and originality	High
College / department	Evaluates the project	Compliance with the mini project index	High
Pharmacy (future)	Would fulfil medicine orders	Verifiable prescriptions	Low
Regulators (future)	Telemedicine and data-protection authorities	Compliance, consent, audit trail	Medium for any real deployment
Cloud provider (Google Firebase)	Hosts identity and data	Fair use of the service within quota	Medium

The register shows that the **patient** is the stakeholder around whom all current requirements are written, while the **doctor**, **pharmacy** and **regulator** appear as future stakeholders. This is the reason a doctor portal, pharmacy integration and compliance features are listed under future scope rather than in the present requirements.

1.5 Feasibility Study

Feasibility was assessed from three viewpoints before design began. Each was judged against the actual resources of a single student developer: one laptop, a free cloud account and roughly one academic semester.

1.5.1 Technical Feasibility

The question is whether the required functionality can be built with technology that is available and well understood. The decisive choice was the backend. Three alternatives were compared.

Table 1.5: Backend alternatives considered

Criterion	Firebase (chosen)	Node.js + Express + MongoDB	PHP + MySQL
Own server to host and maintain	Not required	Required	Required
Authentication	Built in (email/password, hashed passwords)	Must be implemented (hashing, sessions, tokens)	Must be implemented
Per-user data isolation	One security rule on the UID path	Must be coded in every route	Must be coded in every script
Setup effort	Minutes in the Firebase console	Hours: server, database, hosting	Hours: stack and hosting

Criterion	Firebase (chosen)	Node.js + Express + MongoDB	PHP + MySQL
Cost for a prototype	Free tier is sufficient	Hosting cost or local only	Hosting cost or local only
Fit with a single-developer, one-semester project	Very good	Moderate	Moderate

Firebase removes the two parts of a conventional backend that are most error-prone for a beginner - password handling and access control - and delegates them to a managed service. The frontend uses only HTML5, CSS3 and ES6 JavaScript, which are supported by all current browsers; the Firebase v12 SDK is loaded as ES modules and bridged to the classic script. Every feature in the scope table can be implemented with these technologies. The video call and payment are simulated by design, which removes the two hardest integrations (WebRTC and a payment gateway) from the critical path.

Technical risks and their treatment.

Table 1.6: Technical risks

Risk	Likelihood	Treatment
ES module script finishes after the classic script, so Firebase wrappers are missing when the user acts	High	`window` bridge plus `waitForFirebase()` and the `firebase-ready` event
Opening the page from `file://` blocks ES modules	High	Serve the folder over HTTP (local server or hosting)
Remote images become unreachable	Medium	Documented limitation; images may be embedded for offline use
Misconfigured database rules expose user data	Medium	Rules published and tested; documented in Section 5.5

Conclusion. The project is technically feasible.

1.5.2 Economic Feasibility

Economic feasibility compares the cost of building and running the system with the benefit it delivers. Because SmartHealth is an academic prototype, the relevant question is whether it can be completed without spending money beyond what the student already has.

Table 1.7: Estimated cost of the prototype (approximate, INR)

Item	Basis	Estimated cost
Development computer	Already owned by the student	0
Code editor (VS Code) and browser	Free software	0
Firebase Authentication and Realtime Database	Free (Spark) plan is sufficient for a demonstration	0
Hosting (Firebase Hosting or GitHub Pages)	Free tier	0

Item	Basis	Estimated cost
Image assets	Unsplash and Wikimedia Commons (free to use)	0
Custom domain (optional)	Annual registration, only if wanted	Optional, about 800-1,200 per year
Printing and binding of the report	College requirement	As per college

The running cost of the prototype is zero. The free plan of the cloud provider has limits on simultaneous connections, stored data and monthly transfer; these limits are discussed in Section 10.1 and are far above what a classroom demonstration needs. **Benefits** are mainly educational: a complete, demonstrable system that exercises the whole syllabus, and a reusable code base for a possible real implementation. For a production service, costs would rise (paid Firebase plan, a real payment gateway with transaction fees, video infrastructure, compliance and security audits), but these are outside the scope of this project.

Conclusion. The project is economically feasible at zero cost.

1.5.3 Operational Feasibility

Operational feasibility asks whether the system will actually be used and be usable by its intended users, and whether it can be kept running.

- **Ease of use.** The interface has five top-level tabs and no nested menus. A new user can complete a full consultation without instructions, and the user manual in Section 11.2 describes every step.
- **Skills required of the user.** Only the ability to use a web browser. No installation, no configuration.
- **Acceptance.** Patients are already familiar with chat, video calls and UPI-style payments; the application reuses those mental models.
- **Maintainability.** Doctors, triage rules, medicine sets and prices are declared as data structures at the top of the script, so content changes do not require logic changes.
- **Support and operation.** Hosting, availability and backup of the data tier are handled by Firebase, so no operations team is required.
- **Risks.** Over-reliance on the assistant by patients; mitigated by an explicit academic disclaimer in the report and in the application, and by rules that direct urgent symptoms to emergency care.

Conclusion. The project is operationally feasible. Taken together, the three studies show that SmartHealth can be built, can be built at no cost, and can be used and maintained by the people it is meant for.

2. Requirement Engineering

Requirement engineering turns the problem of Section 1 into a precise, testable statement of what the system must do and how well it must do it. Requirements were gathered by four methods: study of existing teleconsultation platforms and symptom checkers, review of the Telemedicine Practice Guidelines and Firebase documentation, discussion with the project guide, and walk-throughs of the intended workflow by the developer acting as a patient. Each requirement is given a unique identifier so that it can be traced to a design element, an implementation module and a test case.

2.1 Functional Requirements Specification

Functional requirements describe the behaviour the system must provide. Table 2.1 lists them with the module that implements each one; the traceability to test cases is given in Section 8.4.

Table 2.1: Functional requirements

ID	Requirement	Module
FR-01	A visitor can create an account using an email address and a password of at least six characters, confirmed twice.	M1
FR-02	A registered user can sign in with the same credentials and is taken to the dashboard.	M1
FR-03	Repeated failed sign-in attempts are counted and the form is locked temporarily after five failures.	M1
FR-04	A signed-in user can send free-text symptom descriptions to the assistant and receive a tailored reply.	M3
FR-05	The assistant is reachable both as a full page and as a floating widget available on every tab.	M3
FR-06	The user can view all available doctors with name, specialty, experience and rating.	M4
FR-07	The user can start a consultation with any listed doctor and end it at will.	M4
FR-08	Ending a consultation generates a prescription whose medicines match the doctor's specialty.	M5
FR-09	The prescription is read-only; the patient cannot add, edit or delete any medicine.	M5
FR-10	The user can pay for the prescription using UPI, Google Pay, PhonePe, Paytm, card or net banking.	M6
FR-11	Payment details are validated per method before the transaction is accepted.	M6
FR-12	Every completed consultation is written to the database and listed newest-first in the history tab.	M7
FR-13	Signing out clears the session and returns the user to the login screen.	M1
FR-14	The application presents five top-level views (Home, AI Assistant, Telemedicine, My Prescriptions, My History) and shows exactly one at a time.	M2
FR-15	Each view that can be empty (prescriptions, history) shows an explanatory empty state.	M5, M7

ID	Requirement	Module
FR-16	During a consultation the user has mic, camera and end-call controls (mic and camera are simulated).	M4

Detailed specification of the core functions

Table 2.2: Input-process-output view of the core functions

Function	Input	Processing	Output
Sign up	Email, password, confirmation	Validate format and length; create account in Firebase; write profile	Dashboard shown; profile stored
Sign in	Email, password	Validate; Firebase check; count failures; lock after five	Dashboard or inline error
Triage	Symptom text	Lower-case; test each rule's keywords in priority order	Advice and recommended specialist
Consultation	Selected doctor	Show connecting state; show live view; wait for end	Session end event
Prescription	Doctor's specialty	Look up medicine set; render sheet	Read-only prescription and medicine array
Payment	Method and details	Validate revealed fields; simulate processing	Receipt with amount, method, transaction ID
History	User ID	Fetch subtree; sort by date descending	Card list or empty state

2.2 Non-Functional Requirements

Non-functional requirements describe the qualities the system must have. They are stated so that each can be checked.

Table 2.3: Non-functional requirements

Attribute	Requirement	How it is checked
Usability	Five top-level tabs, no nested menus. Every irreversible action is confirmed by a visible state change.	Walk-through of the user manual; test T-06 to T-18
Responsiveness	Two breakpoints at 768 px and 420 px. Navigation, doctor grid, prescription header and chat widget reflow for small screens.	Test T-20 at 375 px width
Performance	No build artefacts and no framework runtime. Only network dependencies at load are the Firebase SDK modules and the remote images.	Lighthouse and DevTools (Section 10.1)
Security	Passwords never touch application code; they go directly to Firebase Authentication, which stores only salted hashes. History is written under the caller's own UID.	Rules review and test T-19 (Section 10.3)
Data integrity	The prescription is generated and rendered in one pass with no editable inputs.	Test T-13

Attribute	Requirement	How it is checked
Maintainability	Doctors, rules, medicine sets and prices are data structures at the top of the script.	Code review (Section 11.4)
Reliability	Network failures degrade to a message or an empty history, never to a broken page.	Test T-21
Portability	Runs unchanged on Chrome 90+, Firefox 88+, Edge 90+ and Safari 14+.	Cross-browser tests (Section 8.2)
Accessibility (target)	Readable contrast, visible focus, keyboard-operable forms. Full WCAG conformance is not claimed.	Manual review; gaps listed in Section 11.1
Availability	Depends on the static host and Firebase; no server of our own to fail.	Provider service levels

2.3 Use-Case Analysis

Use cases describe the system from the user's side. The use-case diagram is given in Figure 4.1 (Section 4.1); here each principal use case is specified in text. The only human actor is the **Patient**; **Firebase** and the **simulated payment gateway** are supporting system actors.

Table 2.4: Use case UC-01 - Register account

Item	Description
Actor	Patient (primary); Firebase Authentication (supporting)
Precondition	Patient is not signed in and has opened the application.
Main flow	1. Patient chooses Sign up. 2. Enters email, password and confirmation. 3. System validates the input. 4. Firebase creates the account. 5. System stores the profile (email, creation time) under the new UID. 6. Dashboard is shown.
Alternative flows	3a. Field empty, malformed email, password shorter than six characters or mismatch: inline error, no network call. 4a. Email already registered: translated error message.
Postcondition	A new account exists and the patient is signed in.

Table 2.5: Use case UC-02 - Login and logout

Item	Description
Actor	Patient; Firebase Authentication
Precondition	Patient has an account.
Main flow	1. Patient enters email and password. 2. System validates presence and format. 3. Firebase verifies the credentials. 4. Dashboard is shown. Logout returns to the login screen and clears the session.
Alternative flows	3a. Wrong credentials: attempts counter incremented and remaining attempts shown. 3b. Fifth failure: form locked for 30 seconds. 3c. Network error: message shown, page remains usable.
Postcondition	Patient is authenticated (or the session is cleared on logout).

Table 2.6: Use case UC-03 - Chat with the AI assistant

Item	Description
Actor	Patient
Precondition	Patient is signed in.
Main flow	1. Patient types symptoms in the full-page chat or the floating widget. 2. System escapes the text and shows it. 3. After a short delay the assistant shows the reply of the first matching rule, naming a specialist.
Alternative flows	3a. No rule matches: the fallback asks for onset, severity and associated symptoms. 3b. Input contains HTML: shown as literal text.
Postcondition	Transcript updated; no data is stored.

Table 2.7: Use case UC-04 - Consult a doctor

Item	Description
Actor	Patient; Firebase (history write)
Precondition	Patient is signed in.
Main flow	1. Patient opens Telemedicine and views the doctor list. 2. Chooses a doctor and starts the consultation. 3. System shows a connecting state, then the live view. 4. Patient ends the call. 5. System issues the prescription, opens the prescriptions tab and writes the history record.
Alternative flows	3a. Patient leaves the tab while connecting: live view is not written into the hidden panel. 5a. History write fails: error logged, prescription still shown.
Postcondition	A prescription is displayed and a history entry exists under the patient's UID.

Table 2.8: Use case UC-05 - View prescription

Item	Description
Actor	Patient
Precondition	At least one consultation has been completed in the current session.
Main flow	System shows clinic block, patient block, medicine table, signature block and order button. No field is editable.
Alternative flows	No consultation yet: empty state explains that a consultation must be completed first.
Postcondition	None (read-only).

Table 2.9: Use case UC-06 - Pay for medicines

Item	Description
Actor	Patient; simulated payment gateway
Precondition	A prescription is displayed.
Main flow	1. Patient presses Order / Pay. 2. Modal shows the amount for the specialty. 3. Patient selects a method and enters the method's details. 4. System validates. 5. Processing view for 1.8 s. 6. Receipt with amount, method and transaction ID.

Item	Description
Alternative flows	3a. No method chosen: prompt to select one. 4a. Invalid details: rejected with a message.
Postcondition	Receipt shown; no order record is persisted (simulation).

Table 2.10: Use case UC-07 - View consultation history

Item	Description
Actor	Patient; Firebase Realtime Database
Precondition	Patient is signed in.
Main flow	System fetches the patient's history subtree, sorts by date descending and renders one card per consultation.
Alternative flows	No records: empty state. Network failure: empty history, error logged.
Postcondition	None (read-only).

2.4 Requirement Prioritization

Requirements were prioritised with the **MoSCoW** method so that, had time run short, the most valuable parts of the workflow would still be complete. A requirement is *Must* if the workflow cannot be demonstrated without it, *Should* if it is important but has a workaround, and *Could* if it improves quality but can be dropped.

Table 2.11: MoSCoW prioritisation

Priority	Requirements	Rationale
Must have	FR-01, FR-02, FR-04, FR-06, FR-07, FR-08, FR-09, FR-12, FR-13, FR-14	Without these the patient journey (login, triage, consult, prescription, history) cannot be shown end to end.
Should have	FR-03, FR-10, FR-11, FR-15	Lockout, payment and empty states complete the workflow and its security, but a demonstration is possible without them.
Could have	FR-05, FR-16	The floating widget and mic/camera controls improve convenience and realism.
Won't have (this release)	Doctor portal, real video, real payments, appointment scheduling, notifications	Out of scope; recorded as future scope.

2.5 Constraints and Assumptions

Table 2.12: Constraints

Type	Constraint
Time	The project must be completed within one academic semester (about 14 working weeks).
Team	Single developer; no separate tester or designer.
Budget	No paid services; only free tiers.
Technology	Client-side only; Firebase as the sole backend; no build tools or frameworks.
Legal / ethical	Cannot be presented as a medical device; no real patient data is used; prescriptions are not for treatment.
Platform	Modern browsers with ES module support; internet connection needed for Firebase and for images.

Table 2.13: Assumptions

No.	Assumption
A1	The user has a modern browser and a working internet connection.
A2	The Firebase project stays within the free-plan quota during demonstration.
A3	The Email/Password sign-in provider is enabled and the hosting domain is in the authorised-domains list.
A4	Remote images from Unsplash and Wikimedia remain available.
A5	Users understand that the system is a demonstration and not a medical service.
A6	Each account belongs to one patient; there is no shared or family account.

3. Software Development Life Cycle (SDLC) Planning

SDLC planning decides how the work will be organised in time and effort. For a single-developer, one-semester project the plan has to be simple enough to follow and flexible enough to absorb the things that go wrong, such as the module/classic script problem described in Section 7.1.

3.1 Selection of SDLC Model

Table 3.1: Comparison of candidate SDLC models

Model	Strength	Weakness for this project	Verdict
Waterfall	Simple, well documented	No room to go back; requirements and Firebase behaviour were not fully known at the start	Rejected
Spiral	Explicit risk analysis	Heavy documentation and management overhead for one developer	Rejected
Agile (Scrum)	Fast feedback	Needs a team, roles and ceremonies	Partly borrowed (short iterations)
Prototype	Early working version to review with the guide	Can drift without a fixed requirement baseline	Used together with the model below
Iterative-incremental	Working system grows in steps; each step is tested; feedback fed into the next	Needs discipline to keep increments small	**Selected**

The **iterative-incremental model with prototyping** was selected. The requirement baseline of Section 2 was fixed first (to avoid drift), and the system was then built in increments, each of which ended with a working, testable version that could be shown to the guide.

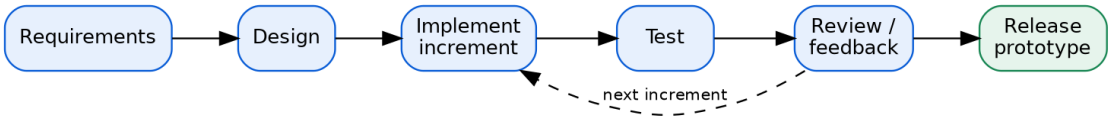


Figure 3.1 – Iterative-incremental model used for SmartHealth

Table 3.2: Planned increments

Increment	Content	Outcome
I-1	Login/sign-up view, Firebase initialisation and bridge, authentication (M1)	User can register, log in and log out
I-2	Dashboard shell, navigation (M2), AI assistant (M3), floating widget	Triage chat works on every tab
I-3	Doctor roster and simulated call (M4), prescription generation (M5)	Consultation produces a read-only prescription

Increment	Content	Outcome
I-4	Payment modal (M6), history persistence and display (M7)	Complete patient journey
I-5	Responsive layout, error handling, hardening, testing, splitting into four files	Release-ready prototype
I-6	Deployment, GitHub repository, final documentation	Deliverables complete

3.2 Work Breakdown Structure (WBS)

The WBS decomposes the project into six work packages, each broken into three tasks that can be estimated, assigned and checked off.

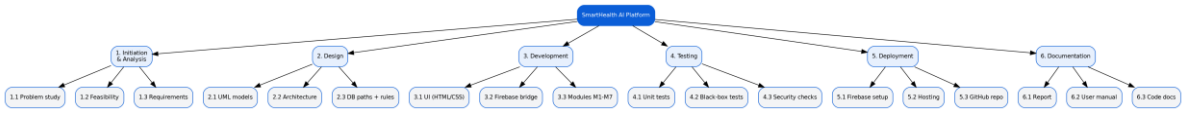


Figure 3.2 – Work breakdown structure of the SmartHealth project

Table 3.3: Work packages and deliverables

WP	Work package	Principal tasks	Deliverable
1	Initiation and analysis	Problem study, feasibility, requirements	Sections 1 and 2
2	Design	UML models, architecture, database paths and rules	Sections 4 and 5
3	Development	UI, Firebase bridge, modules M1-M7	Four source files
4	Testing	Unit, black-box, integration, security checks	Section 8 and 10 results
5	Deployment	Firebase setup, hosting, GitHub repository	Live URL and repository
6	Documentation	Technical report, user manual, code documentation	Section 11

3.3 Project Timeline

The plan covers 14 working weeks. Documentation runs in parallel with the other work from week 3, so that design decisions are written down while they are fresh.

Table 3.4: Project timeline and milestones

Weeks	Phase	Milestone at the end of the phase
1-2	Problem study and feasibility	M-A: problem statement and feasibility approved by guide
2-3	Requirement engineering	M-B: requirement baseline (FR/NFR) frozen
3-4	SDLC planning	M-C: WBS, timeline and resource plan complete
4-7	UML modelling and architecture	M-D: design review with guide
6-12	Implementation (increments I-1 to I-4)	M-E: complete patient journey working

Weeks	Phase	Milestone at the end of the phase
11-13	Integration and system testing	M-F: all test cases executed, defects logged
12-14	Deployment and GitHub	M-G: hosted application and repository
3-14	Documentation (parallel)	M-H: final report submitted

3.4 Gantt Chart

Figure 3.3 shows the same plan as a Gantt chart. Bars that overlap show activities carried out in parallel. The **critical path** runs through requirement engineering, architecture design, implementation of the Firebase bridge and authentication, the remaining modules and then testing; a slip in any of these delays the whole project, whereas documentation has float.

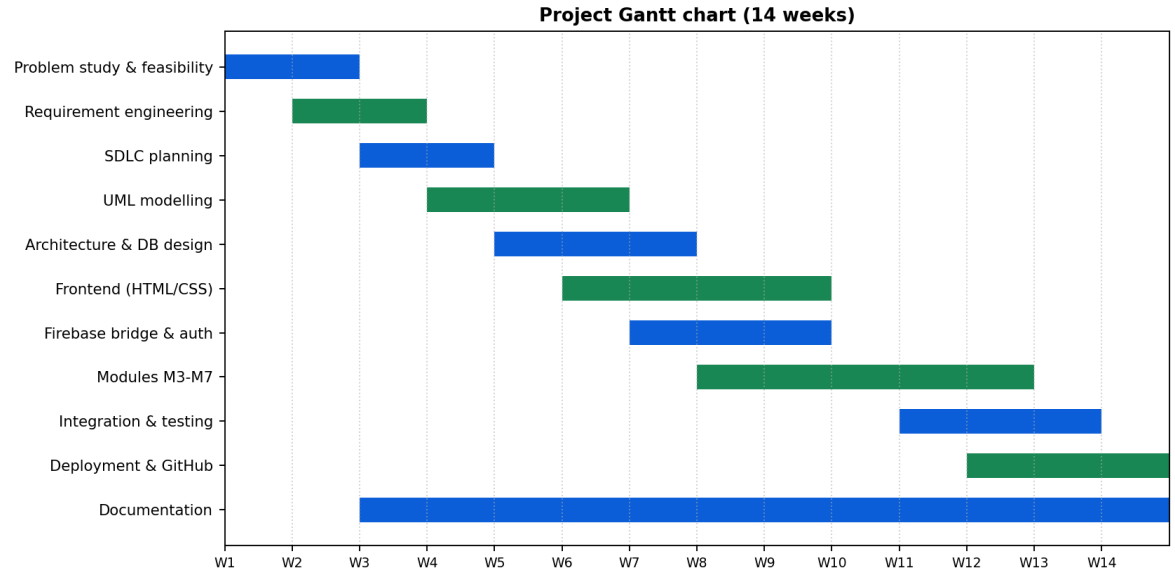


Figure 3.3 – Gantt chart of the 14-week plan

3.5 Resource Planning

Table 3.5: Human resources

Role	Person	Responsibility
Developer, analyst, tester, documenter	Student (project author)	All technical work and the report
Project guide	Prof. Sabina Pathan	Periodic review, guidance on design and report
Examiner	College appointed	Evaluation

Table 3.6: Hardware, software and cloud resources

Category	Resource	Purpose
Hardware	Laptop (dual-core 1.6 GHz or better, 4 GB RAM recommended)	Development and testing
Hardware	Smartphone	Responsive-layout testing

Category	Resource	Purpose
Software	VS Code (or any editor), Chrome / Firefox / Edge, Git	Coding, testing, version control
Software	Python 3 (`http.server`) or VS Code Live Server	Local hosting
Cloud	Firebase project: Authentication, Realtime Database, Hosting	Backend and deployment
Cloud	GitHub account	Repository and optional GitHub Pages hosting
Content	Unsplash and Wikimedia Commons	Portraits and backdrops

Table 3.7: Risk plan

Risk	Impact	Mitigation
Firebase configuration error	Login fails	Test in isolation first (increment I-1)
Scope creep (real video, payments)	Delay	Scope fixed in Section 1.3; items moved to future scope
Loss of source files	Rework	Git repository with regular commits
Unavailable remote images	Blank panels	Fallback colours; option to embed images
Time shortage near submission	Incomplete documentation	Documentation written in parallel from week 3

4. System Modeling Using UML

The Unified Modeling Language was used to describe the system from several viewpoints before coding: **functional** (use case), **static structure** (class, ER), **behavioural** (sequence, activity) and **physical** (deployment). Because SmartHealth is written in JavaScript with module-level functions rather than classes, the class diagram shows *logical* classes - the concepts that the code manipulates - and the functions that operate on them.

4.1 Use Case Diagram

Figure 4.1 shows the use cases offered to the patient. Dashed arrows labelled *include* show behaviour that is always performed as part of another use case: registering or logging in always validates the credentials with Firebase, ending a consultation always generates the prescription and always saves the history record, and starting a consultation always requires the doctor list. The supporting actors on the right are **Firebase** (authentication and database) and the **simulated payment gateway**.

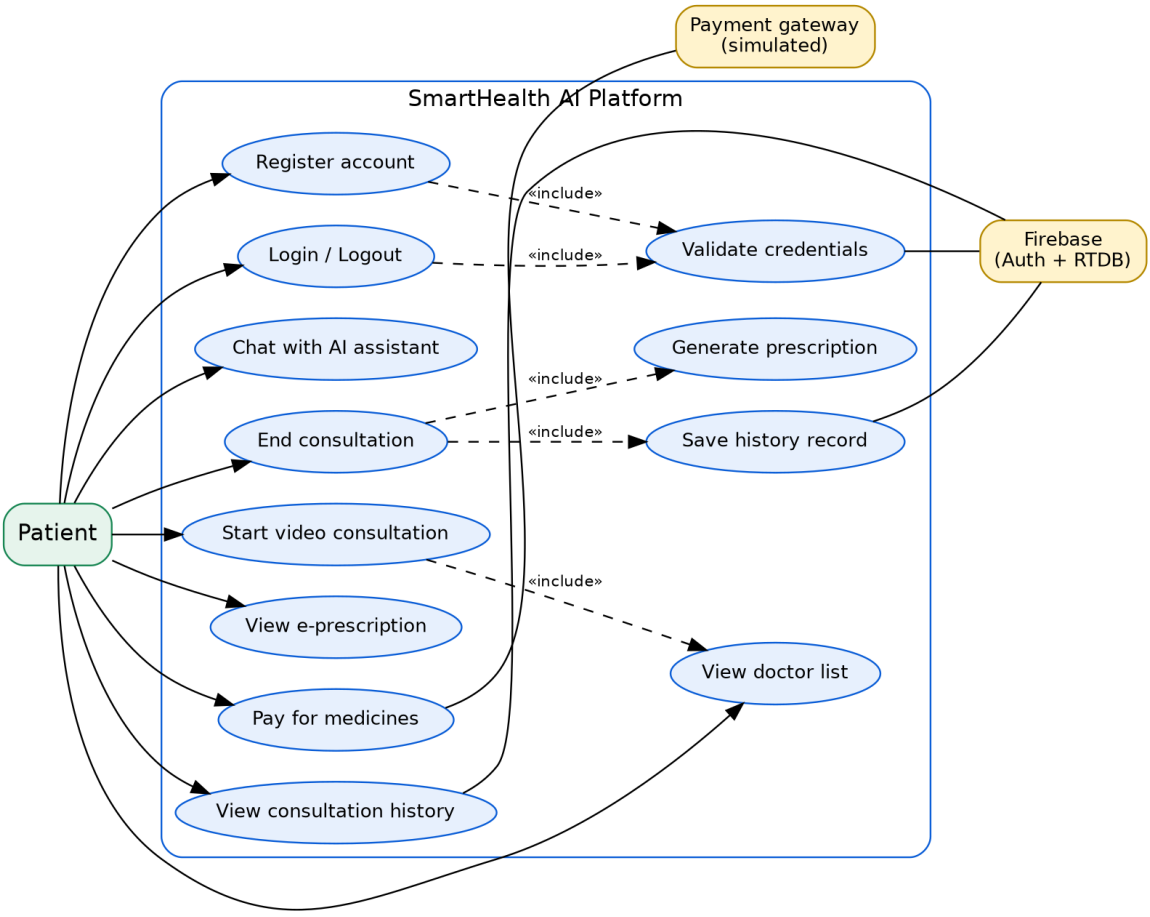


Figure 4.1 – Use case diagram of SmartHealth

Table 4.1: Use cases and requirement traceability

Use case	Requirements covered
Register account	FR-01

Use case	Requirements covered
Login / Logout	FR-02, FR-03, FR-13
Chat with AI assistant	FR-04, FR-05
View doctor list	FR-06
Start / End consultation	FR-07, FR-16
View e-prescription	FR-08, FR-09, FR-15
Pay for medicines	FR-10, FR-11
View consultation history	FR-12, FR-15

4.2 Class Diagram

The logical classes of the system and their relationships are shown in Figure 4.2. A **User** starts many **Consultations**; each consultation is attended by one **Doctor** and produces exactly one **Prescription**, which contains one or more **Medicines**. Each consultation is recorded as one **HistoryRecord**, owned by the user. A prescription can be paid for through a **Payment** (zero or one per prescription). The **AIAssistant** is used by the user and is independent of the other classes: it stores nothing.

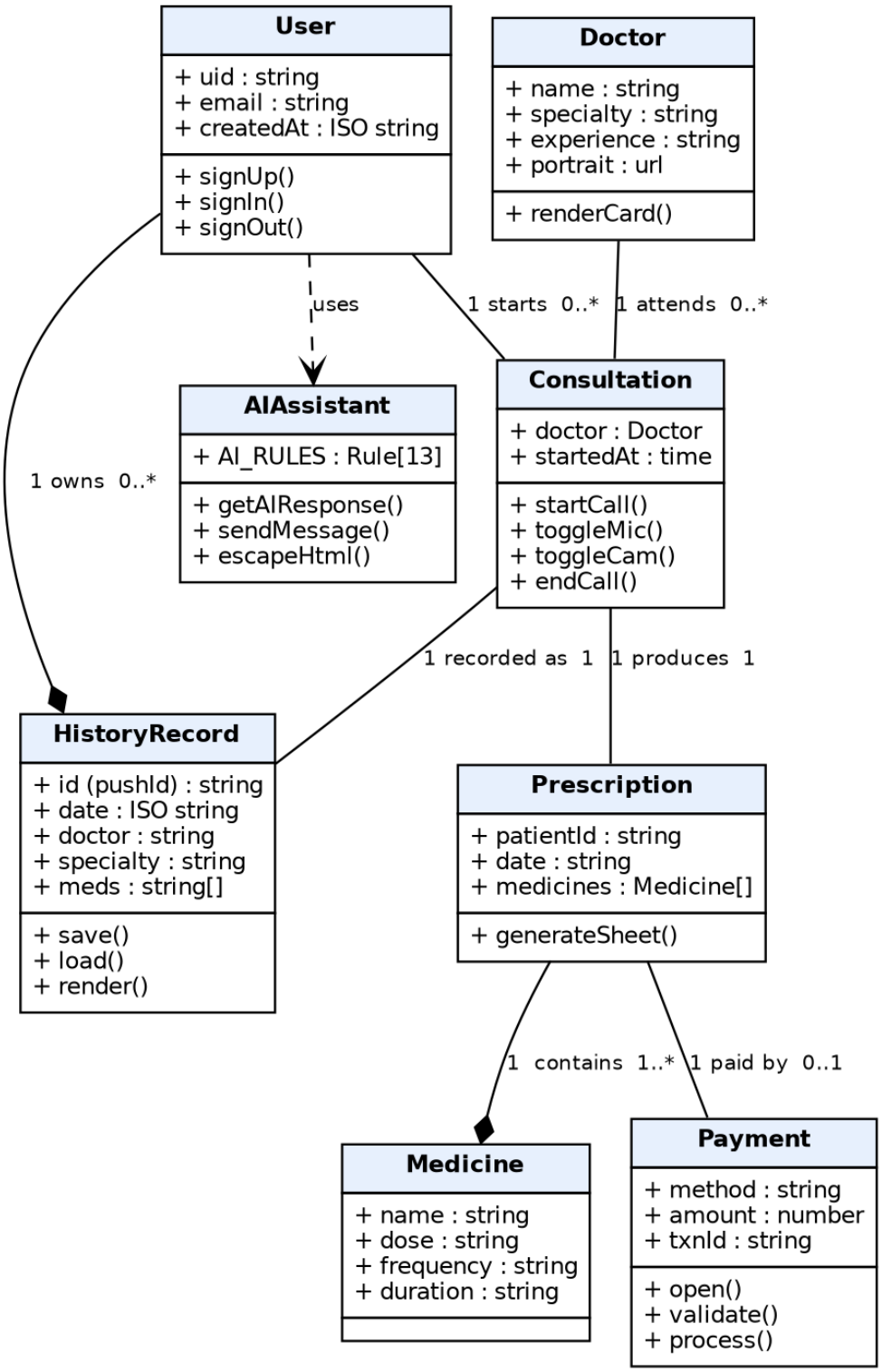


Figure 4.2 – Class diagram (logical classes)

Table 4.2: Class responsibilities and implementing code

Class	Responsibility	Implemented by
User	Identity and session	`handleAuth`, `completeLogin`, `logout`, Firebase Authentication
Doctor	Static roster entry	`DOCTORS` array, `renderDoctorGrid`

Class	Responsibility	Implemented by
Consultation	Session between patient and doctor	`startCall`, `toggleMic`, `toggleCam`, `endCall`
Prescription	Read-only sheet and medicine list	`getMedsForSpecialty`, `generatePrescriptionSheet`
Medicine	Name, dose, frequency, duration	Objects returned by `getMedsForSpecialty`
HistoryRecord	Persisted summary of a consultation	`saveHistoryEntry`, `loadHistory`, `renderHistory`
Payment	Method, amount, transaction ID	`openPaymentModal`, `processPayment`, `computePrescriptionAmount`
AIAssistant	Rule-based triage	`AI_RULES`, `getAIResponse`, `sendMessage`, `sendTabMessage`, `escapeHtml`

4.3 Sequence Diagram

Three sequence diagrams show the most important interactions in time order. Solid arrows are calls; dashed arrows are returns.

Sign-in sequence

The browser validates the form, waits until the Firebase bridge exists, asks Firebase Authentication to verify the credentials and either completes the login or records a failed attempt. The password is passed to Firebase and never stored by application code.

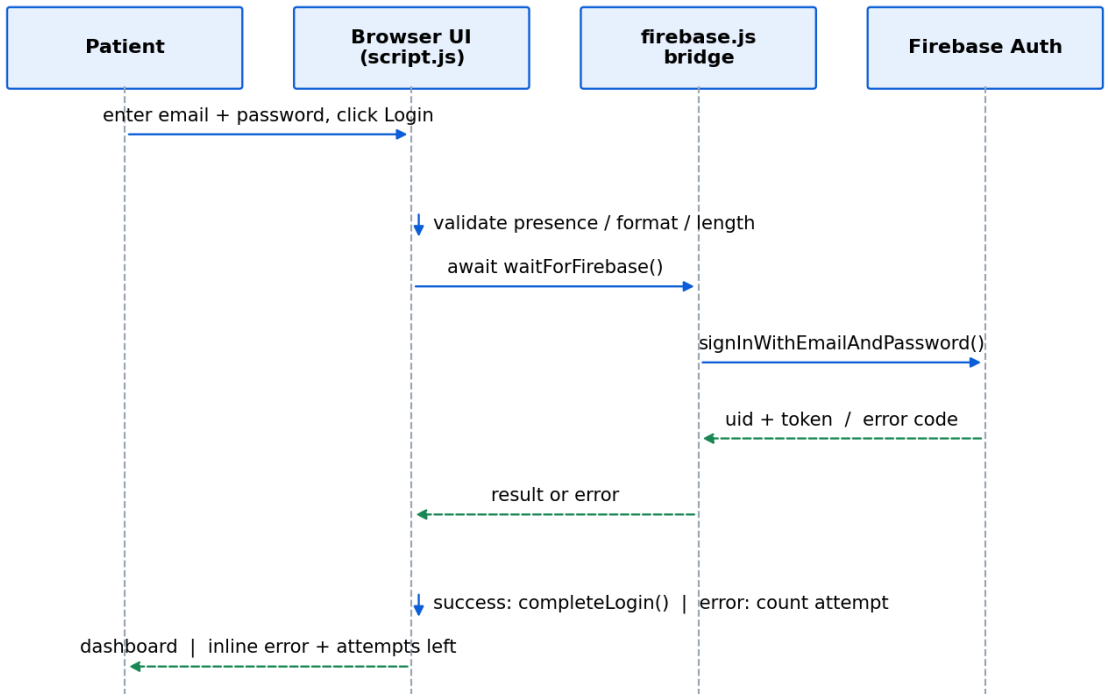


Figure 4.3 – Sequence diagram: sign-in

Consultation and prescription sequence

This is the central workflow. It is the only path that writes to the database. The history record is written from the same medicine array that was rendered, so the history can never contradict the prescription.

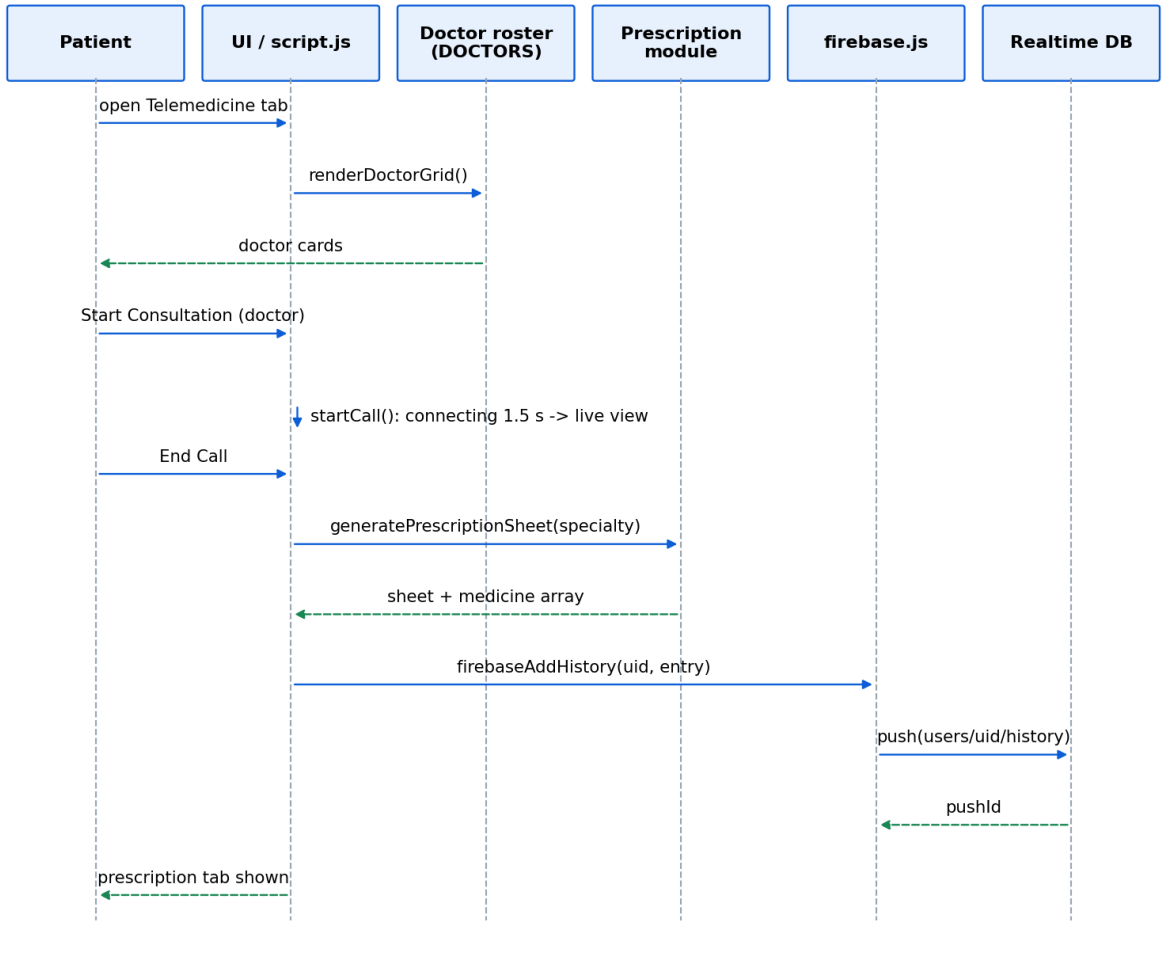


Figure 4.4 – Sequence diagram: consultation and prescription

Payment sequence

The payment modal always starts from a reset state, so a second payment never inherits state from the first. Validation is per method; processing and the receipt are simulated.

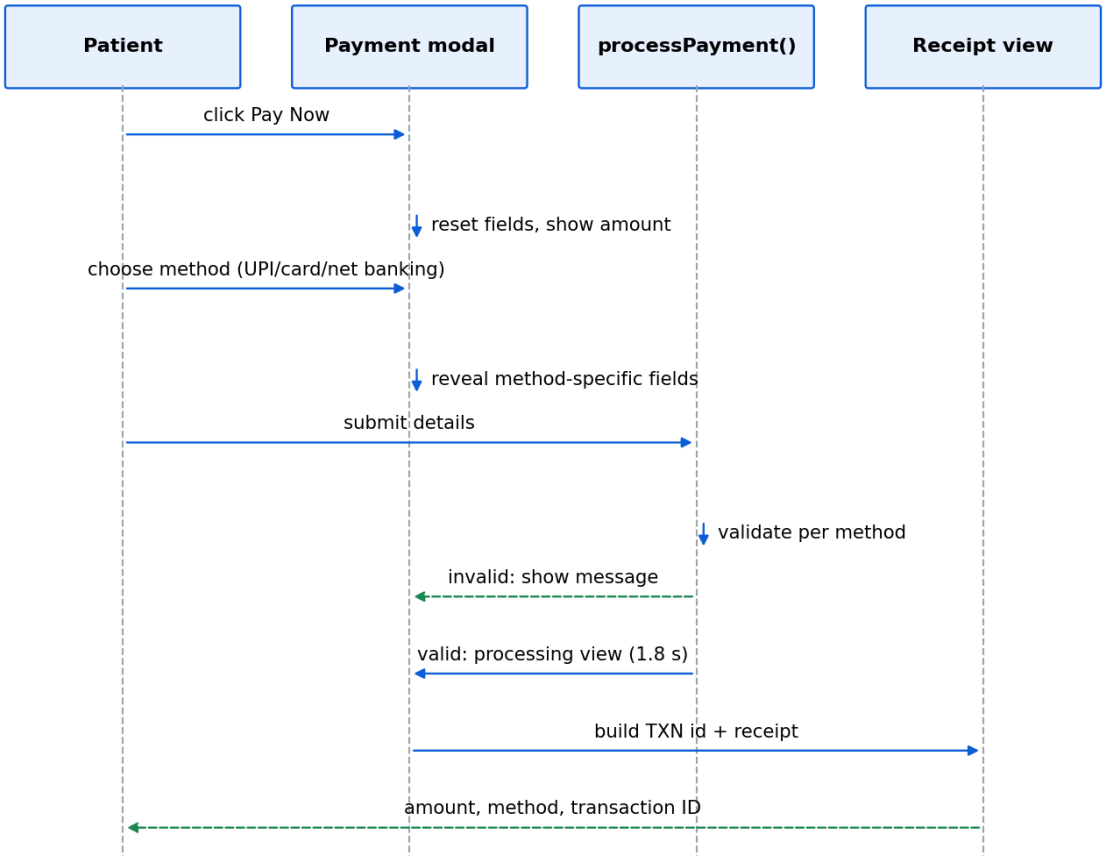


Figure 4.5 – Sequence diagram: medicine payment

4.4 Activity Diagram

Figure 4.6 shows the activity flow of a complete consultation, from doctor selection to payment, as decisions and actions. Figure 4.7 shows the authentication activity flow including the lockout loop.

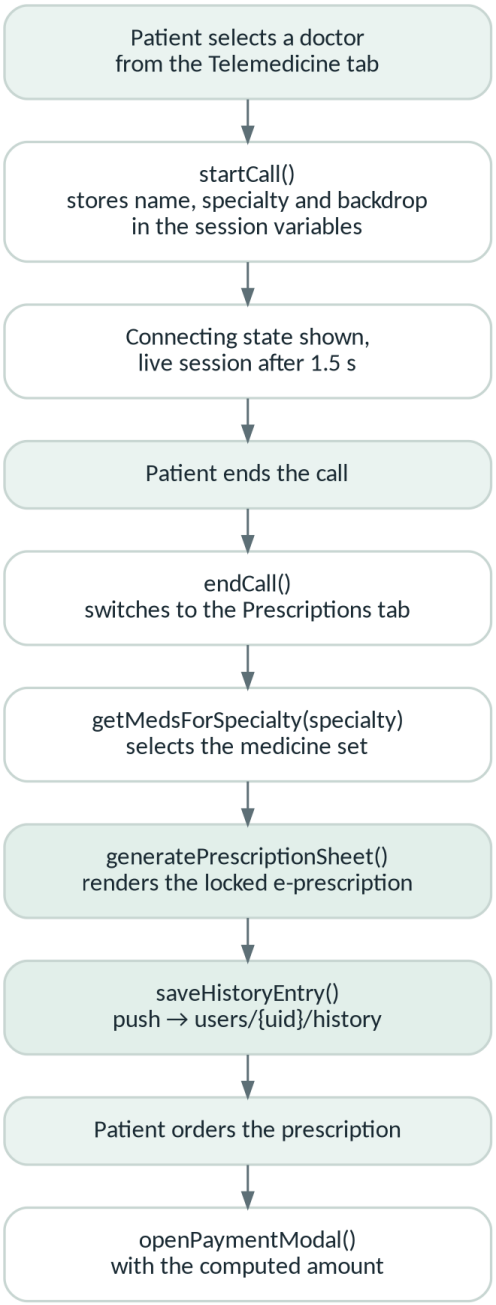


Figure 4.6 – Activity diagram: consultation, prescription and payment

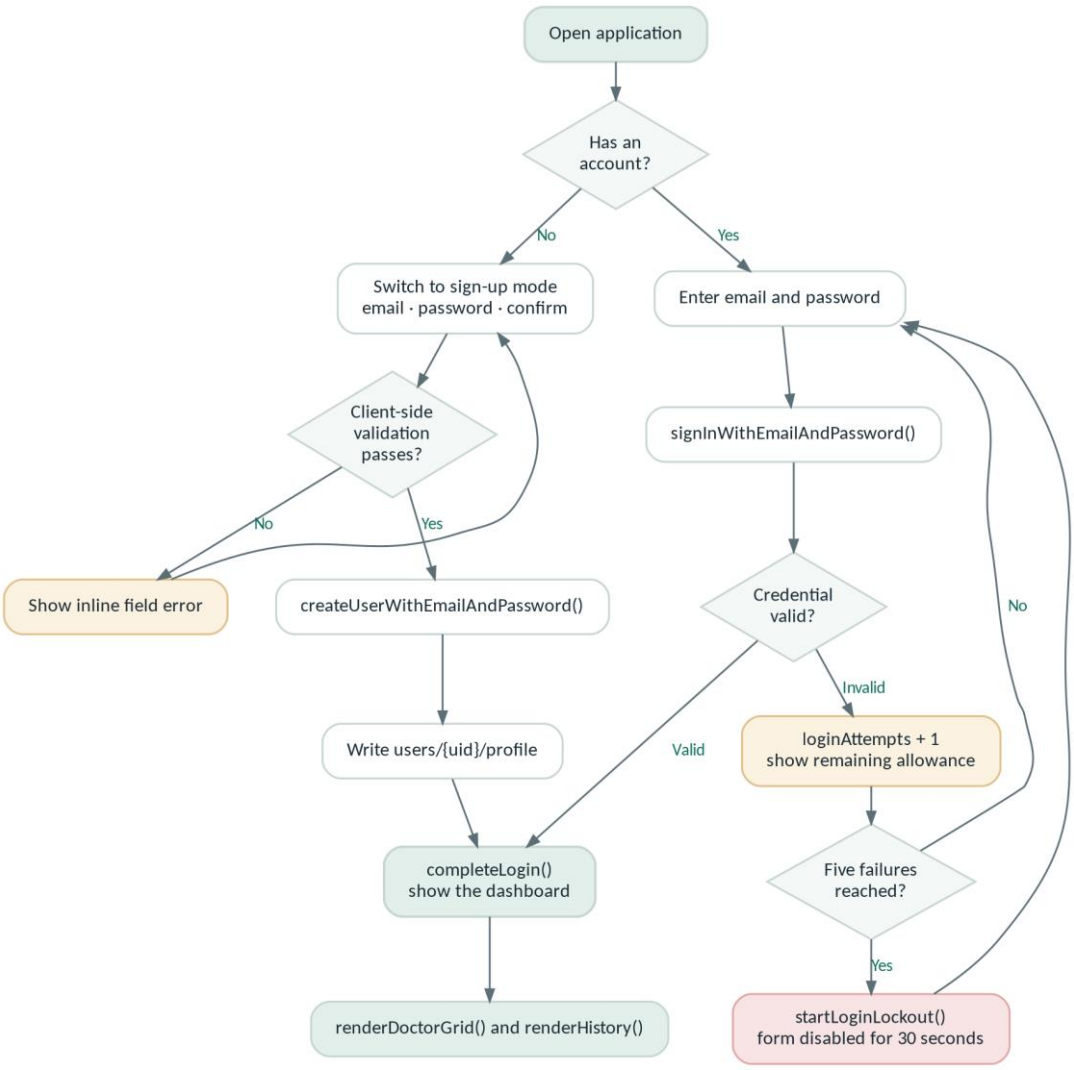


Figure 4.7 – Activity diagram: authentication and lockout

In the authentication flow, the decision after the Firebase call has three exits: success (dashboard), a recoverable error (inline message and attempt counter) and the lockout branch on the fifth failure, which disables the form for thirty seconds before restoring it. The consultation flow has its only decision at the payment stage, where invalid input returns the patient to the same modal.

4.5 Entity Relationship (ER) Diagram

Firebase Realtime Database is a JSON tree, not a relational store, so the ER diagram in Figure 4.8 is a **logical** model: it shows the entities and the relationships that the JSON tree encodes. Entities with a blue header are persisted in Firebase; entities with a yellow header are held as static data in the script.

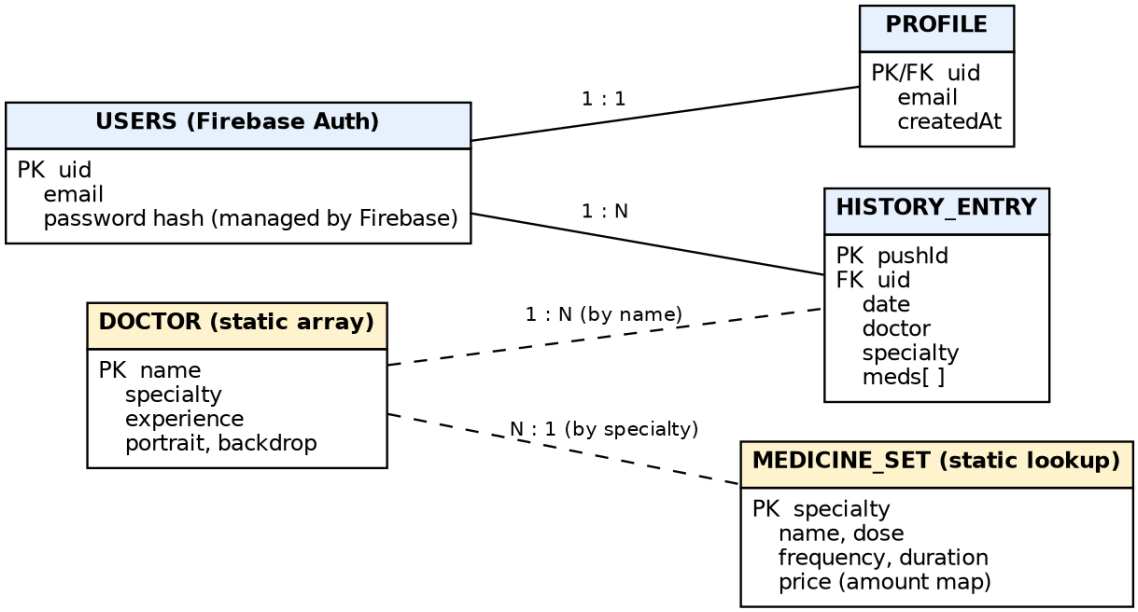


Figure 4.8 – Logical ER diagram

Table 4.3: Entities, keys and storage

Entity	Key	Attributes	Stored in
USERS	uid	email, password hash	Firebase Authentication
PROFILE	uid (one-to-one with USERS)	email, createdAt	Realtime Database: users/{uid}/profile
HISTORY_ENTRY	pushId	date, doctor, specialty, meds[ ]	Realtime Database: users/{uid}/history/{pushId}
DOCTOR	name	specialty, experience, portrait, backdrop	`DOCTORS` array in script.js
MEDICINE_SET	specialty	name, dose, frequency, duration; price per specialty	`getMedsForSpecialty` and price map in script.js

Normalisation note. In a relational design the history entry would hold foreign keys to doctor and medicine tables. In a JSON tree, reading one patient's history must be one fast, self-contained read, so the entry copies the doctor's name, specialty and the medicine names (**denormalisation**). This also means a historical record stays correct if the static roster later changes.

4.6 Deployment Diagram

Figure 4.9 shows the physical arrangement. The patient's browser downloads the four static files from a static host over HTTPS, then talks directly to Firebase for authentication and data. Remote images are fetched from public image hosts. There is no application server of our own.

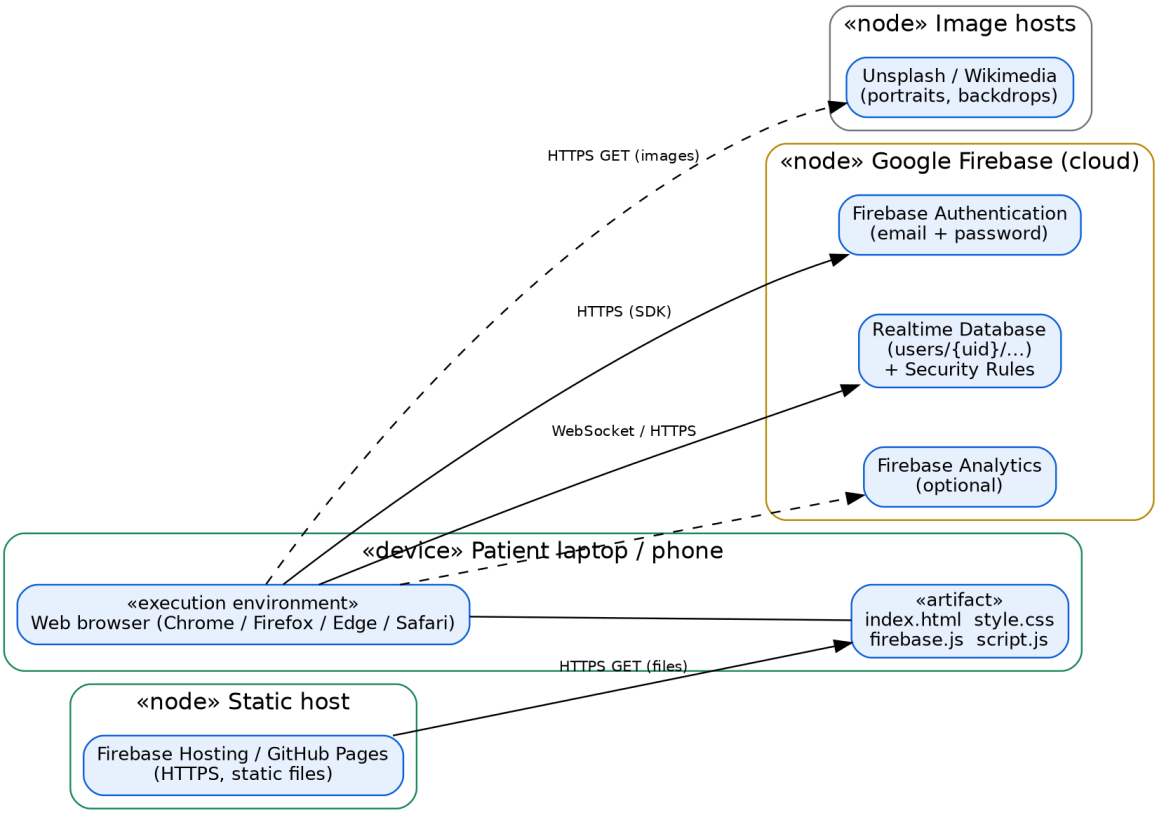


Figure 4.9 – Deployment diagram

Table 4.4: Deployment nodes

Node	Hosts	Protocol
Patient device	Browser executing index.html, style.css, firebase.js and script.js	HTTPS
Static host	Firebase Hosting or GitHub Pages serving the files	HTTPS GET
Firebase Authentication	Email/password identity	HTTPS via SDK
Firebase Realtime Database	JSON tree and security rules	WebSocket / HTTPS via SDK
Image hosts	Unsplash and Wikimedia portraits, backdrops and signature	HTTPS GET

5. System Architecture Design

This section describes how the system is organised internally. Having a precise architecture is especially important in a serverless application, because with no server of our own every rule about who may read what must be enforced either in the browser or by Firebase.

5.1 Frontend Architecture

The frontend is a single-page application made of three layers, shown in Figure 5.1: the **presentation layer** (`index.html` and `style.css`), the **behaviour layer** (`script.js`, a classic script that implements modules M1-M7) and the **integration layer** (`firebase.js`, an ES module that talks to Firebase and exposes wrapper functions on `window`).

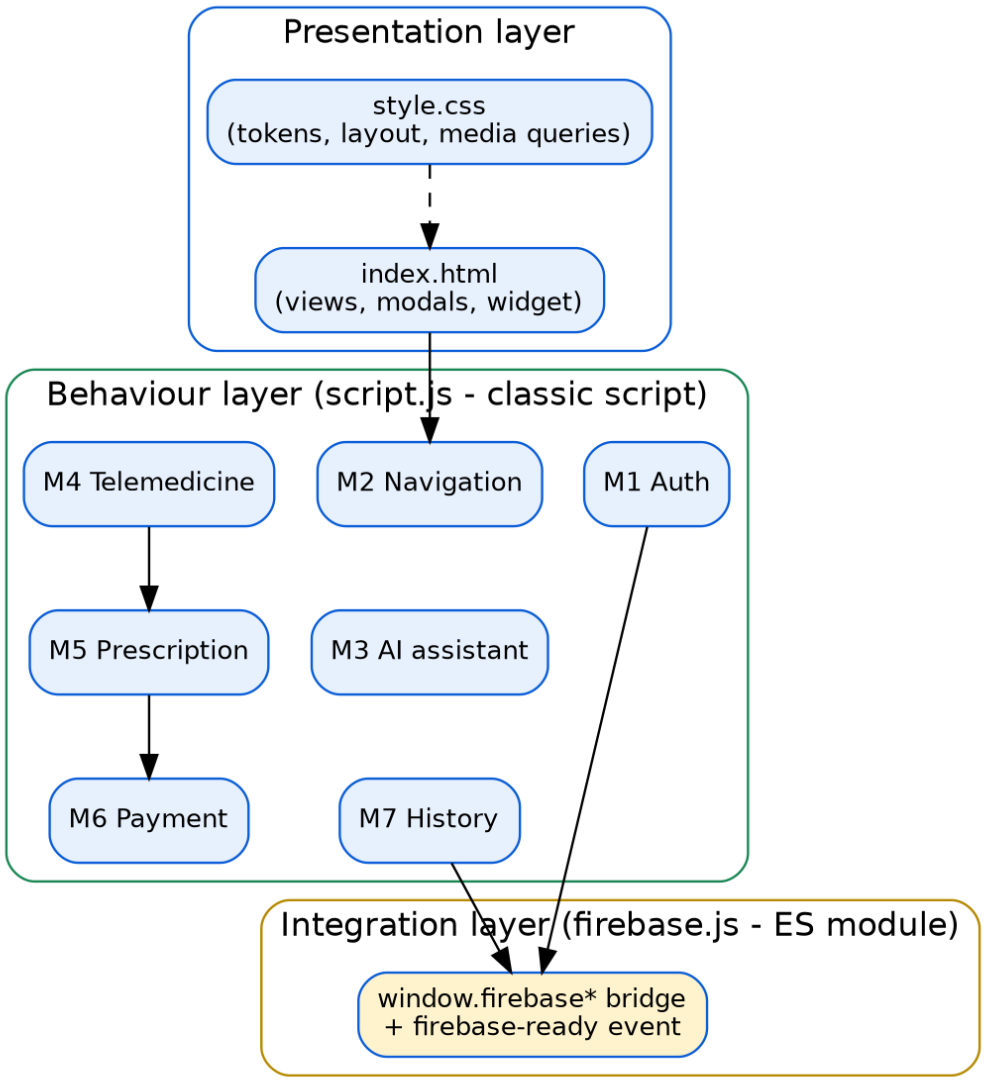


Figure 5.1 – Layers of the frontend

Views and routing

All five views exist in the DOM at the same time and exactly one carries the `active` class. `switchTab()` removes that class everywhere, adds it to the requested view and mirrors

the change on the navigation buttons. The telemedicine tab always resets to the doctor list rather than resuming an old call, and the history tab re-fetches from the database so a consultation recorded moments ago is visible.

Session state

Table 5.1: Principal module-level state variables

Variable	Meaning
authMode	Whether the single auth form is in sign-in or sign-up mode
loginAttempts	In-memory count of failed sign-ins
selected doctor variables (three)	Doctor name, specialty and portrait of the active call
window.firebaseCurrentUser	Current user, kept up to date by `onAuthStateChanged`
window.firebaseReady	True once the Firebase module has finished loading

Visual system

Colour is centralised in CSS custom properties on `:root`. The primary blue `#0b5ed7` carries navigation, headings and primary actions; `#198754` marks success and purchase; `#dc3545` marks sign-out and ending a call. Surfaces use two shadow tokens and a shared 12 px radius. Layout uses Flexbox and Grid, with media queries at 768 px and 420 px.

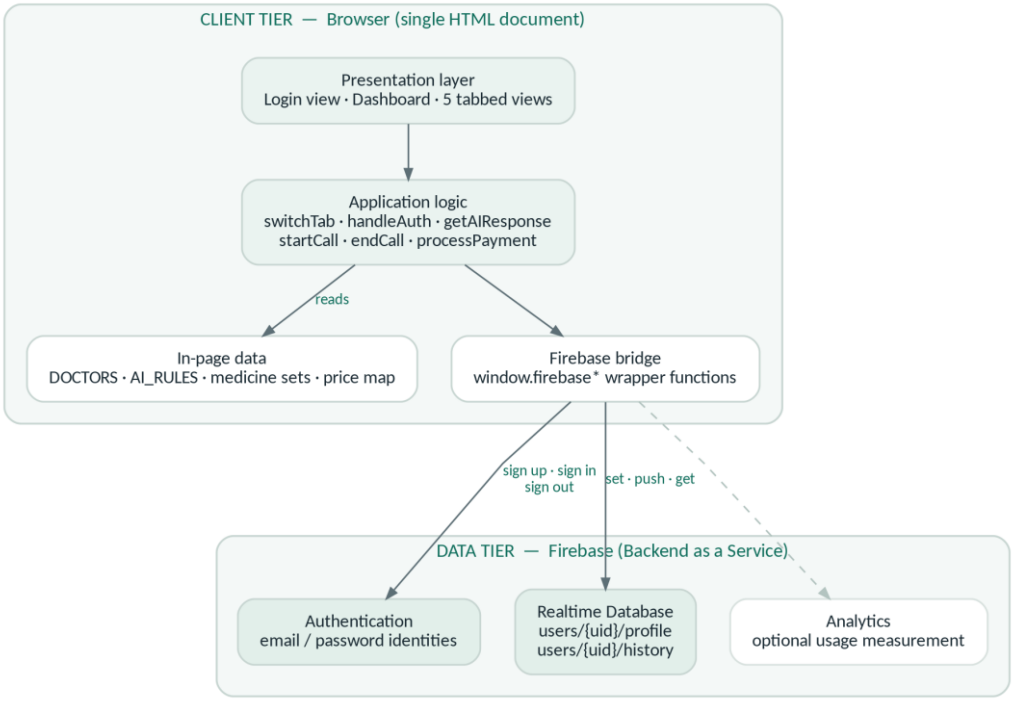


Figure 5.2 – Two-tier architecture. The ES module script exposes Firebase calls on window so the classic script can reach them.

Data flow

The context-level data flow is shown in Figure 5.3. The patient supplies credentials, symptoms, a doctor choice and payment details; the system returns advice, a prescription, a

receipt and the history; Firebase receives authentication requests and history entries and returns authentication results and records.

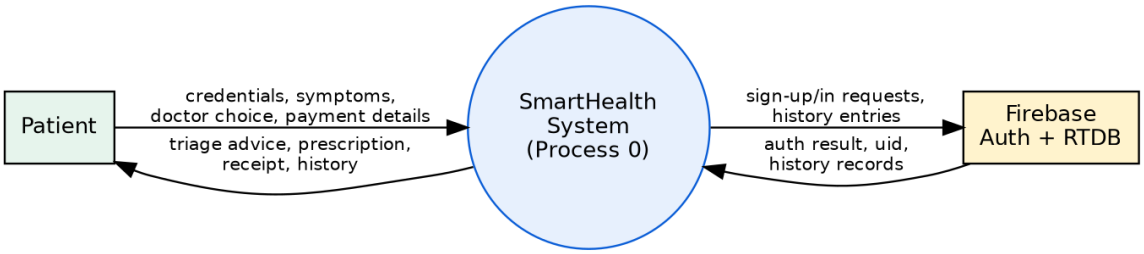


Figure 5.3 – Context-level data flow diagram

Website navigation

Figure 5.4 shows how a signed-in patient moves between the sections of the portal. Registration and login lead to the dashboard, which exposes the five tabs; the floating assistant is reachable from every tab, and the payment modal is opened only from the prescription sheet.

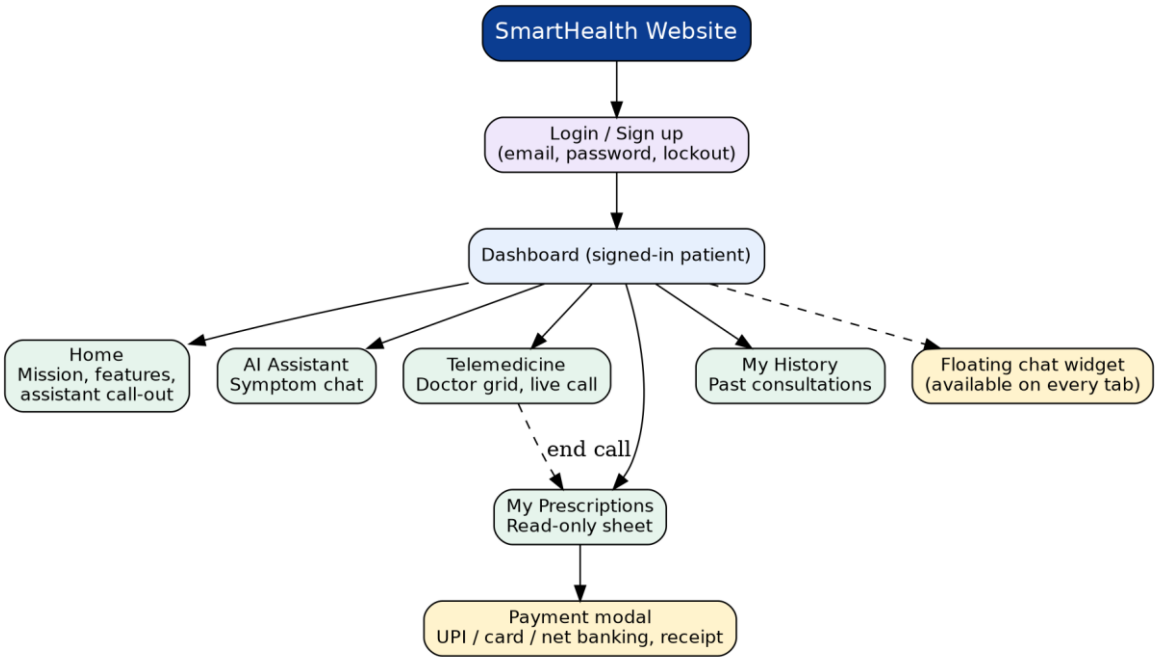


Figure 5.4 – Website navigation of SmartHealth

5.2 Backend Architecture

SmartHealth uses Firebase as a **backend-as-a-service**. The browser reaches Firebase through the bridge in `firebase.js`; Authentication verifies identities and Realtime Database stores data, with Security Rules deciding who may read or write each path.

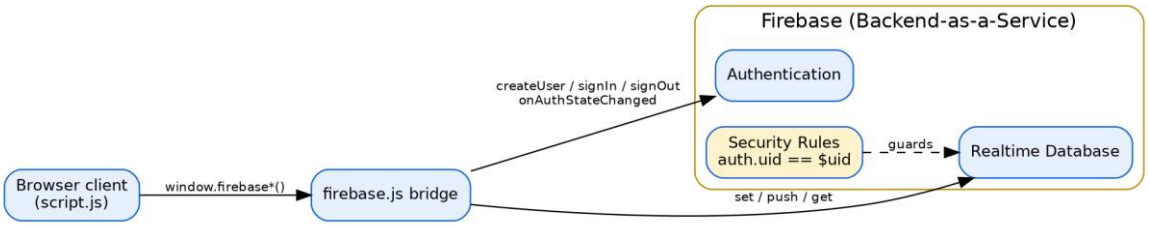


Figure 5.5 – Backend architecture

Table 5.2: Module decomposition

#	Module	Responsibility	Principal functions
M1	Authentication	Sign-up, sign-in, validation, attempt lockout, sign-out	handleAuth, toggleAuthMode, completeLogin, startLoginLockout, logout
M2	Navigation	Show one view, hide the rest, sync the active nav button	switchTab
M3	AI assistant	Keyword triage, two chat surfaces, output escaping	getAIResponse, sendMessage, sendTabMessage, escapeHtml
M4	Telemedicine	Doctor roster rendering, call session lifecycle	renderDoctorGrid, startCall, toggleMic, toggleCam, endCall
M5	Prescription	Specialty-to-medicine mapping and sheet rendering	getMedsForSpecialty, generatePrescriptionSheet
M6	Payment	Method selection, per-method validation, transaction simulation	openPaymentModal, selectPaymentMethod, processPayment, computePrescriptionAmount
M7	History	Write and read consultation records, chronological rendering	saveHistoryEntry, loadHistory, renderHistory

Firebase Analytics is initialised as optional telemetry; any failure to start it is caught and ignored so that it can never affect the application.

5.3 Database Schema Design

Because Realtime Database is a schemaless JSON tree, the design question is not "which tables" but "which paths". The whole tree is rooted at `users` and partitioned by Firebase UID, which makes per-user isolation expressible as a single security rule.

```
{
  "users": {
    "<uid>": {
      "profile": {
        "email": "patient@example.com",
        "createdAt": "2026-09-19T10:24:31.204Z"
      },
      "history": {
        "<pushId>": {
          "date": "2026-09-19T11:02:48.771Z",
          "doctor": "Dr. John Doe",
          "specialty": "Cardiologist",
          "meds": ["Aspirin 75mg", "Atorvastatin 20mg"]
        }
```

```
        }
      }
    }
}
```

Table 5.3: Field reference

Path	Type	Written by	Description
users/{uid}/profile/email	string	Sign-up	Lower-cased email; also shown as the patient ID on the prescription
users/{uid}/profile/createdAt	ISO string	Sign-up	Account creation timestamp
users/{uid}/history/{pushId}	object	End of call	One consultation record; the key is a Firebase push ID
...{pushId}/date	ISO string	End of call	Consultation timestamp, used for the descending sort
...{pushId}/doctor	string	End of call	Name of the consulting doctor
...{pushId}/specialty	string	End of call	Specialty, which determines the medicine set
...{pushId}/meds	string array	End of call	Names of the prescribed medicines

Why push IDs

History entries are created with `push()` rather than `set()` at a chosen key. A push ID is generated from a timestamp plus randomness, so it is unique without coordination and sorts chronologically by construction. Two consultations created in the same millisecond, or from two devices at once, cannot overwrite each other.

5.4 API Structure

SmartHealth exposes no HTTP API of its own. Its "API" is the internal contract between the two scripts: thin wrapper functions that `firebase.js` assigns to `window` and that `script.js` calls. This keeps all Firebase imports in one file.

Table 5.4: Bridge API between script.js and firebase.js

Bridge item	Purpose	Underlying Firebase call
firebaseCreateAccount(email, password)	Create an account	createUserWithEmailAndPassword
Sign-in wrapper	Verify credentials	signInWithEmailAndPassword
Sign-out wrapper	End the session	signOut
firebaseAddHistory(uid, entry)	Append a consultation record	push(ref(db, users/{uid}/history), entry)
History-read wrapper	Fetch the patient's history subtree once	get(ref(db, users/{uid}/history))
firebaseCurrentUser	Currently signed-in user	Updated by onAuthStateChanged
firebaseReady / `firebase-ready` event	Signals that the wrappers exist	Set after initialisation

Table 5.5: Database operations and the rule that guards them

Operation	Path	Method	Guard
Write profile	users/{uid}/profile	set	auth.uid == $uid
Append history	users/{uid}/history	push	auth.uid == $uid
Read history	users/{uid}/history	get (once)	auth.uid == $uid

5.5 Security Considerations

With no application server, the client is the only component talking to the database, so the **Security Rules are the actual access-control layer** and must be published together with the code.

```
{
  "rules": {
    "users": {
      "$uid": {
        ".read":  "auth != null && auth.uid == $uid",
        ".write": "auth != null && auth.uid == $uid"
      }
    }
  }
}
```

With these rules an authenticated patient can read and write only the subtree named after their own UID. A request for another user's path is rejected by Firebase itself, not by the browser.

Table 5.6: Threats and countermeasures

Threat	Countermeasure
Password theft from the application	Passwords are passed directly to Firebase Authentication; only salted hashes are stored by the provider; application code never keeps them
Reading or altering another patient's data	UID-scoped Security Rules enforced server-side by Firebase
Script injection through chat text	Every string is passed through `escapeHtml()` (textContent -> innerHTML) before being added to the transcript
Account enumeration	`auth/wrong-password` and `auth/invalid-credential` map to one identical message
Brute-force sign-in	Attempt counter with a 30-second lockout in the UI, plus Firebase's own `auth/too-many-requests` response
Tampering with a prescription	The sheet has no form controls; no patient-facing code path mutates a medicine
Use of the API key by others	The key identifies the project, not a user; protection comes from rules and the authorised-domains list

6. Coding Of Website

This chapter documents the source code of SmartHealth. The application is a client-side web application made of four files. There is no server of our own: the "backend" is Google Firebase, which is configured from its console, so backend coding consists of the Firebase initialisation and bridge (`firebase.js`), the data-access functions that use it, and the database security rules. Key excerpts of each file are listed below; the complete files are submitted with the project and are kept in the GitHub repository described in Section 9.5.

> **About the listings**
> The listings above are the principal parts of each file. Identifiers such as the sign-in and sign-out wrapper names follow the naming used in this report; the complete source files, which are the reference, are submitted with the project and stored in the GitHub repository.

6.1 Frontend coding

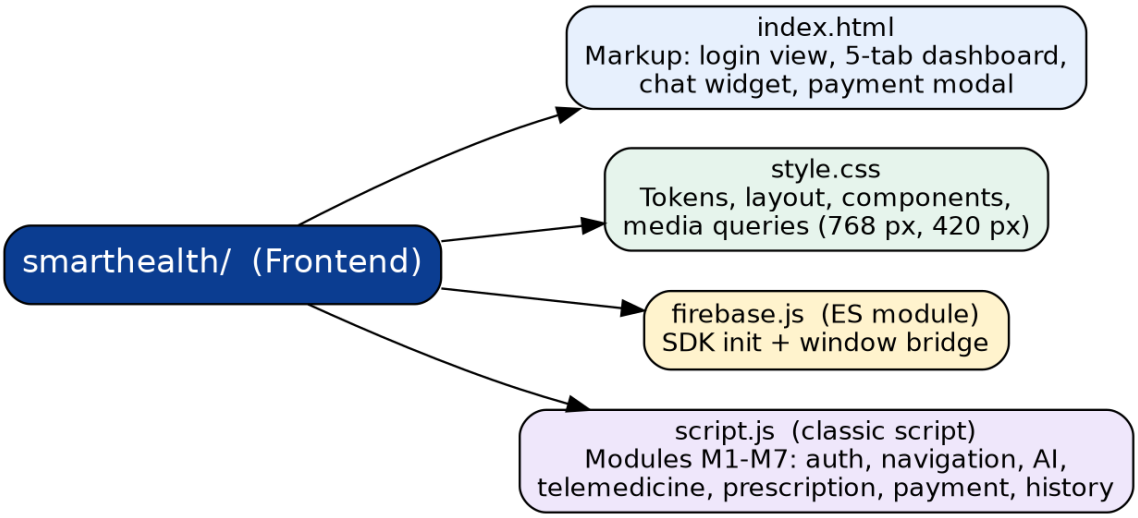


Figure 6.1 – Frontend structure of SmartHealth

6.1.1 index.html

`index.html` contains the markup of every view. All views are present in the page from the start and are shown or hidden with a CSS class. The skeleton below shows the structure of the file.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>SmartHealth AI Platform</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <!-- 1. Login / sign-up view (single form, two modes) -->
  <section id="login-view"> ... email, password, confirm password, button ...
</section>
```

```
  <!-- 2. Dashboard with five tabs -->
  <div id="dashboard" style="display:none">
    <nav> Home | AI Assistant | Telemedicine | My Prescriptions | My History |
Sign out </nav>
    <section class="tab active" id="tab-home">          ... </section>
    <section class="tab"        id="tab-ai">            ... </section>
    <section class="tab"        id="tab-telemedicine">  ... doctor grid, call
panel ... </section>
    <section class="tab"        id="tab-prescriptions"> ... </section>
    <section class="tab"        id="tab-history">       ... </section>
  </div>

  <!-- 3. Floating chat widget (available on every tab) -->
  <div id="chat-widget"> ... </div>

  <!-- 4. Payment modal -->
  <div id="payment-modal"> ... methods, conditional fields, receipt ... </div>

  <script type="module" src="firebase.js"></script>
  <script src="script.js"></script>
</body>
</html>
```

6.1.2 style.css

The stylesheet defines the design tokens on `:root`, the layout and the two responsive breakpoints.

```
:root {
  --primary: #0b5ed7;   /* navigation, headings, primary actions */
  --success: #198754;   /* success and purchase */
  --danger:  #dc3545;   /* sign-out and ending a call */
  --radius:  12px;      /* shared corner radius */
}
html, body { overflow-x: hidden; }
.tab        { display: none; }
.tab.active { display: block; }

/* diagonal watermark on the prescription sheet */
.rx-sheet { position: relative; }
.rx-sheet::before {
  content: "CONFIDENTIAL MEDICAL RECORD";
  position: absolute; inset: 0; display: grid; place-items: center;
  transform: rotate(-30deg); opacity: .06; pointer-events: none;
}

@media (max-width: 768px) { /* nav wraps, doctor grid = 1 column, widget full
width */ }
@media (max-width: 420px) { /* smaller portraits and login heading */ }
```

6.1.3 script.js

`script.js` holds the data, the logic of modules M1-M7 and the event handling. Principal excerpts follow.

Data declared at the top of the file

```
const MAX_LOGIN_ATTEMPTS = 5;
const LOCKOUT_SECONDS   = 30;

const DOCTORS = [
```

```
  { name: "Dr. John Doe", specialty: "Cardiologist", experience: "...", portrait:
"...", backdrop: "..." },
  // ... eight doctors in total, one object each
];

const PRICES = {
  "Cardiologist": 685, "Orthopedic Surgeon": 560, "Dermatologist": 420,
  "General Physician": 340, "Gynecologist": 390, "ENT Specialist": 360,
  "Pediatrician": 300, "Psychiatrist": 250
};
```

Navigation (M2)

```
function switchTab(name) {
  document.querySelectorAll('.tab').forEach(t => t.classList.remove('active'));
  document.getElementById('tab-' + name).classList.add('active');
  document.querySelectorAll('nav button').forEach(b =>
    b.classList.toggle('active', b.dataset.tab === name));
  if (name === 'telemedicine') showDoctorList();   // never resume an old call
  if (name === 'history')      renderHistory();    // re-fetch from the database
}
```

AI assistant (M3)

```
function escapeHtml(text) {
  const d = document.createElement('div');
  d.textContent = text;
  return d.innerHTML;
}

function getAIResponse(message) {
  const text = message.toLowerCase();
  for (const rule of AI_RULES) {              // order = clinical priority
    if (rule.keywords.some(k => text.includes(k))) return rule.reply;
  }
  return FALLBACK_REPLY;
}
```

Telemedicine and prescription (M4, M5)

```
function startCall(doctor) {
  selectedDoctor = doctor;
  showConnecting();
  setTimeout(() => {
    if (callPanelIsDisplayed()) showLiveView(doctor);   // guard against tab
changes
  }, 1500);
}

function endCall() {
  clearVideo();
  switchTab('prescriptions');
  const meds = generatePrescriptionSheet(selectedDoctor.specialty);
  saveHistoryEntry(selectedDoctor, meds);
}

function getMedsForSpecialty(specialty) {
  switch (specialty) {
    case 'Cardiologist':       return ['Aspirin 75mg', 'Atorvastatin 20mg'];
    case 'Dermatologist':      return ['Cetirizine 10mg', 'Hydrocortisone cream
1%'];
    case 'Orthopedic Surgeon': return ['Ibuprofen 400mg', 'Calcium with Vitamin
D3'];
```

```
    /* ... five more branches ... */
    default:                    return ['Paracetamol 500mg', 'Azithromycin 250mg',
'Herbal cough syrup'];
  }
}

function computePrescriptionAmount(specialty) {
  return PRICES[specialty] ?? 340;           // default amount
}
```

Payment validation (M6)

```
function processPayment() {
  const method = selectedMethod;
  if (!method) return showPayError('Please select a payment method.');
  if (['upi','gpay','phonepe','paytm'].includes(method)) {
    const id = upiInput.value.trim();
    if (!id.includes('@') || id.length < 5) return showPayError('Invalid UPI
ID.');
  } else if (method === 'card') {
    const num = cardNumber.value.replace(/\s+/g, '');
    if (num.length < 12 || !expiry.value.includes('/') ||
!/^\d{3}$/.test(cvv.value))
      return showPayError('Invalid card details.');
  } else if (method === 'netbanking' && !bankSelect.value) {
    return showPayError('Please select a bank.');
  }
  showProcessing();
  setTimeout(() => showReceipt('TXN' + String(Date.now()).slice(-10)), 1800);
}
```

6.2 Backend coding

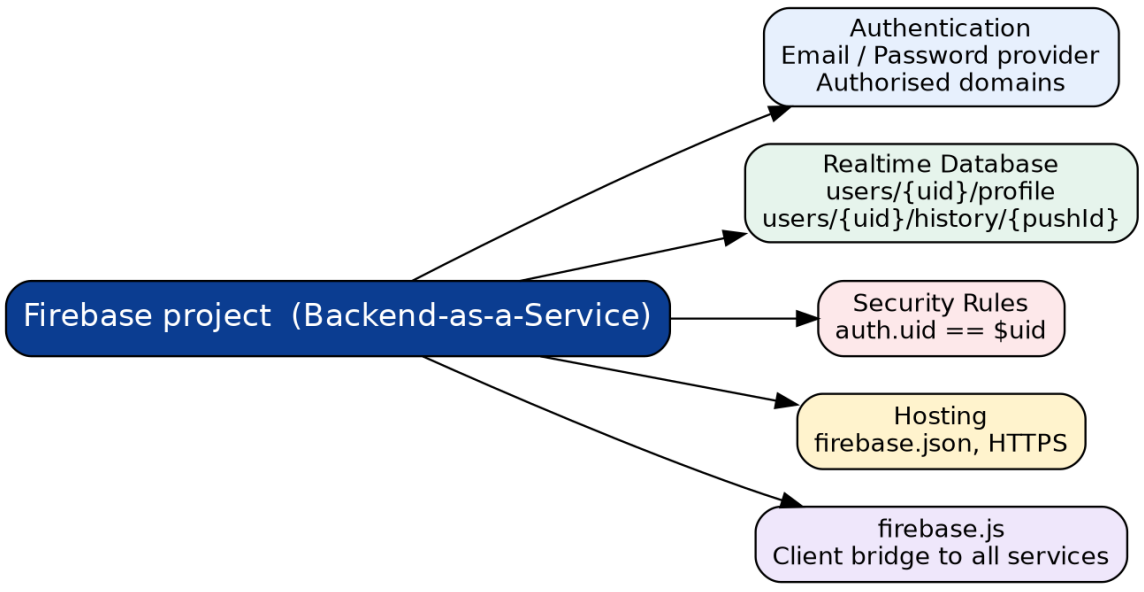


Figure 6.2 – Backend structure (Firebase services)

6.2.1 firebase.js

`firebase.js` is loaded as an ES module. It initialises Firebase and publishes wrapper functions on `window`, so that the classic script can use them.

```
import { initializeApp } from "firebase/app";
import { getAuth, createUserWithEmailAndPassword, signInWithEmailAndPassword,
         signOut, onAuthStateChanged } from "firebase/auth";
import { getDatabase, ref, push, get, set } from "firebase/database";

const firebaseConfig = { apiKey: "...", authDomain: "...", databaseURL: "...",
                         projectId: "...", appId: "..." };   // from the Firebase
console

const app  = initializeApp(firebaseConfig);
const auth = getAuth(app);
const db   = getDatabase(app);

window.firebaseCreateAccount = (email, password) =>
  createUserWithEmailAndPassword(auth, email, password);
window.firebaseSignIn  = (email, password) => signInWithEmailAndPassword(auth,
email, password);
window.firebaseSignOut = () => signOut(auth);
window.firebaseSetProfile = (uid, profile) => set(ref(db,
`users/${uid}/profile`), profile);
window.firebaseAddHistory = (uid, entry)   => push(ref(db,
`users/${uid}/history`), entry);
window.firebaseGetHistory = (uid)          => get(ref(db,
`users/${uid}/history`));

onAuthStateChanged(auth, user => { window.firebaseCurrentUser = user; });

window.firebaseReady = true;
window.dispatchEvent(new Event("firebase-ready"));
```

6.2.2 Database access functions (script.js)

```
async function waitForFirebase() {
  if (window.firebaseReady) return;
  await new Promise(r => window.addEventListener('firebase-ready', r, { once:
true }));
}

async function saveHistoryEntry(doctor, meds) {
  const user = window.firebaseCurrentUser;
  if (!user) return;
  try {
    await window.firebaseAddHistory(user.uid, {
      date: new Date().toISOString(),
      doctor: doctor.name, specialty: doctor.specialty, meds
    });
  } catch (err) { console.error(err); }
}

async function loadHistory() {
  const user = window.firebaseCurrentUser;
  if (!user) return [];
  try {
    const snap = await window.firebaseGetHistory(user.uid);
```

```
    return Object.entries(snap.val() || {})
      .map(([id, rec]) => ({ id, ...rec }))
      .sort((a, b) => new Date(b.date) - new Date(a.date));
  } catch (err) { console.error(err); return []; }
}
```

6.2.3 Database security rules

```
{
  "rules": {
    "users": {
      "$uid": {
        ".read":  "auth != null && auth.uid == $uid",
        ".write": "auth != null && auth.uid == $uid"
      }
    }
  }
}
```

6.2.4 firebase.json

```
{
  "hosting": {
    "public": ".",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"]
  }
}
```

7. Application Development

7.1 Frontend Implementation

The frontend is implemented in three files that link together through ordinary `<link>` and `<script>` tags. `index.html` holds the markup of the login view, the dashboard with its five tabs, the floating chat widget, the payment modal and the footer. `style.css` holds the design tokens, layout, components and media queries. `script.js` holds the application logic. `firebase.js` is loaded as `<script type="module">`.

```
smarthealth/
  index.html    Markup: login view, dashboard (5 tabs), chat widget,
                payment modal, footer; links the other three files
  style.css     Custom properties, layout, components, media queries
  firebase.js   <script type="module">  Firebase init + window bridge
  script.js     <script> classic       Application logic (modules M1-M7)
```

Page structure

All views are present in the DOM from the start and are shown or hidden by toggling a CSS class. This makes tab switching instantaneous and avoids a client-side router. The login view is shown first; after a successful login the dashboard replaces it.

Styling

```
:root {
  --primary: #0b5ed7;   /* navigation, headings, primary actions */
  --success: #198754;   /* success and purchase */
  --danger:  #dc3545;   /* sign-out and ending a call */
  --radius:  12px;      /* shared corner radius */
}
@media (max-width: 768px) { /* nav wraps, doctor grid = 1 column, widget full
width */ }
@media (max-width: 420px) { /* smaller portraits and login heading */ }
```

At 768 px the navigation centres and wraps onto two rows, the doctor grid collapses to a single column, the video panel shortens from 450 px to 260 px, the chat widget spans the viewport width, the prescription header stacks and its table scrolls horizontally inside its own container. A second breakpoint at 420 px reduces portraits and the login heading. Because the medicine table scrolls inside its wrapper and `html, body` carry `overflow-x: hidden`, the page itself never scrolls sideways on a phone.

AI assistant implementation

The assistant is a deterministic rules engine, not a machine-learning model. `AI_RULES` is an array of thirteen objects, each pairing a list of trigger keywords with a written reply. `getAIResponse()` lower-cases the input and returns the reply of the first rule with any keyword contained in the text.

```
function getAIResponse(message) {
  const text = message.toLowerCase();
  for (const rule of AI_RULES) {
    if (rule.keywords.some(k => text.includes(k))) return rule.reply;
  }
```

```
  return fallbackReply;   // asks for onset, severity, associated symptoms
}
```

Rule order encodes clinical priority. The chest-pain and breathing-difficulty rule is first so that "I have a fever and chest pain" resolves to the urgent response rather than the fever response. Every rule ends by naming the specialist on the platform who should see the patient, which connects triage to the telemedicine tab. The rule for mental health points to crisis services and the rule for prescription requests refuses outright, because issuing medication is a doctor's action in this system and not the assistant's.

Table 7.1: Representative categories covered by the rule set

Category	Behaviour of the reply
Urgent chest pain / breathing difficulty (first rule)	Urgent-care advice and cardiology recommendation
Fever, cold and cough, headache	Self-care guidance and recommendation of a general physician
Skin, bones and joints, ear-nose-throat complaints	Recommendation of the matching specialist
Children and women's health	Recommendation of pediatrician or gynecologist
Mental health	Supportive reply that points to crisis services
Request for a prescription	Refusal: only a doctor issues medication
No match (fallback)	Asks for onset, severity and associated symptoms

The same engine drives two surfaces, the full-page chat and the floating widget, through `sendTabMessage()` and `sendMessage()`. Both append the patient's message, clear the input, scroll to the bottom and add the reply after a 700 ms delay that reads as the assistant composing a response. Both pass every string through `escapeHtml()`, which assigns text to a detached element's `textContent` and reads back its `innerHTML`.

Telemedicine, prescription and payment screens

Eight doctors are declared in the `DOCTORS` array, each with a name, specialty, experience string, portrait and consultation backdrop; `renderDoctorGrid()` maps the array into cards, so adding a specialist means adding one object. Starting a call records the doctor in three session variables, hides the roster, reveals the call panel and shows a connecting state; after 1.5 s the panel is replaced by the live layout with backdrop, self-view inset and a red live badge. The replacement is guarded by a check that the call panel is still displayed.

`getMedsForSpecialty()` returns the medicine set through an eight-branch lookup with the General Physician set as default. `generatePrescriptionSheet()` composes the document: clinic block, patient block, the Rx symbol, the medicine table, a signature block and the order button. A diagonal confidential-record watermark is drawn with a CSS `::before` pseudo-element and `pointer-events: none`.

Table 7.2: Medicine set per specialty

Specialty	Medicines issued
General Physician (default)	Paracetamol 500 mg; Azithromycin 250 mg; Herbal cough syrup
Cardiologist	Aspirin 75 mg; Atorvastatin 20 mg
Dermatologist	Cetirizine 10 mg; Hydrocortisone cream 1%
Orthopedic Surgeon	Ibuprofen 400 mg; Calcium with Vitamin D3
Psychiatrist	Melatonin 3 mg
Pediatrician	Paracetamol syrup; Oral rehydration salts
Gynecologist	Folic acid 5 mg; Mefenamic acid 250 mg
ENT Specialist	Azithromycin 250 mg; Saline nasal spray

> **Why the prescription is read-only**
> The read-only guarantee is structural: the sheet contains no form controls of any kind, only table cells and one button that opens the payment modal. There is no patient-facing code path that mutates a medicine. The function also returns the medicine array so that `endCall()` can persist exactly what was displayed.

Table 7.3: Consultation price map (used by the payment module)

Specialty	Amount	Specialty	Amount
Cardiologist	Rs. 685	Gynecologist	Rs. 390
Orthopedic Surgeon	Rs. 560	ENT Specialist	Rs. 360
Dermatologist	Rs. 420	Pediatrician	Rs. 300
General Physician	Rs. 340	Psychiatrist	Rs. 250

7.2 Backend Implementation

There is no backend code of our own. Backend implementation therefore consists of three activities: creating and configuring the Firebase project, initialising the SDK in `firebase.js`, and bridging it to the classic script.

Firebase project set-up

1. Create a project in the Firebase console and register a web app to obtain the configuration object.
2. In Authentication, enable the **Email/Password** sign-in provider.
3. Create a **Realtime Database** and publish the security rules of Section 5.5.
4. In Authentication, Settings, add `localhost` and later the deployment domain to the **authorised domains**.

The Firebase bridge

The Firebase v12 SDK is a set of ES modules, which can only be imported from a `<script type="module">`. The rest of the application is a classic script, so it cannot import them directly. The bridge solves this by importing the SDK in `firebase.js`, initialising the app and assigning thin wrapper functions onto `window`.

```
import { initializeApp } from "firebase/app";
import { getAuth, createUserWithEmailAndPassword,
         signInWithEmailAndPassword, signOut, onAuthStateChanged } from
"firebase/auth";
import { getDatabase, ref, push, get, set } from "firebase/database";

const app  = initializeApp(firebaseConfig);   // config object from the console
const auth = getAuth(app);
const db   = getDatabase(app);

window.firebaseCreateAccount = (email, password) =>
  createUserWithEmailAndPassword(auth, email, password);
window.firebaseAddHistory = (uid, entry) =>
  push(ref(db, `users/${uid}/history`), entry);
onAuthStateChanged(auth, u => { window.firebaseCurrentUser = u; });

window.firebaseReady = true;
window.dispatchEvent(new Event("firebase-ready"));
```

Module scripts are deferred, so they finish after the classic script has been parsed. A user who clicks the login button very quickly could call a wrapper that does not exist yet. `waitForFirebase()` closes that race: if the flag is already set it returns immediately, otherwise it awaits the `firebase-ready` event once.

```
async function waitForFirebase() {
  if (window.firebaseReady) return;
  await new Promise(resolve =>
    window.addEventListener('firebase-ready', resolve, { once: true }));
}
```

> **Lesson from the file split**
> An early attempt to split the single HTML file into separate files failed because the Firebase functions were not visible to the classic script. The window bridge and the `firebase-ready` event are the fix, and are the reason the delivered project has exactly four files that link together and work.

7.3 Database Integration

The history module is the only part of the application that uses the database after sign-up. `saveHistoryEntry()` pushes a record containing the timestamp, doctor, specialty and medicine names. It reads the current user from `window.firebaseCurrentUser` and returns silently if no one is signed in.

`loadHistory()` fetches the subtree once, converts the keyed object into an array with `Object.entries()`, folds each push ID into the record as `id` and sorts by date descending so

the most recent consultation appears first. `renderHistory()` then renders either the card list or the empty state.

```
async function loadHistory() {
  const user = window.firebaseCurrentUser;
  if (!user) return [];
  try {
    const snap = await window.firebaseGetHistory(user.uid);
    const data = snap.val() || {};
    return Object.entries(data)
      .map(([id, rec]) => ({ id, ...rec }))
      .sort((a, b) => new Date(b.date) - new Date(a.date));
  } catch (err) { console.error(err); return []; }
}
```

Order of operations at the end of a call is deliberate: the video element is cleared, the patient is told the prescription is being issued, the prescriptions tab is opened, the sheet is generated, and only then the history record is written from the medicine list that was rendered.

7.4 Authentication and Validation

A single function, `handleAuth()`, serves both sign-up and sign-in; the module-level variable `authMode` decides which. `toggleAuthMode()` flips that variable and rewrites the subtitle, button label, prompt text and the visibility of the confirm-password field, so one form does the work of two.

Three validation layers

1. **Presence and format, in the browser.** Empty fields, a failed email regular expression, a password under six characters on sign-up, or a mismatched confirmation are reported inline with the offending field outlined in red.
2. **Firebase.** The credential is then checked by Firebase Authentication, which is the only component that ever sees the password hash.
3. **Error translation.** `firebaseErrorMessage()` maps Firebase error codes onto readable sentences.

Table 7.4: Firebase error codes and the messages shown

Firebase error code	Meaning	Message to the user
auth/email-already-in-use	Email is registered	An account with this email already exists
auth/invalid-email	Malformed email	Enter a valid email address
auth/weak-password	Password too weak	Password must be at least 6 characters
auth/invalid-credential / auth/wrong-password	Wrong credentials	Same single message for both - does not reveal whether the email exists
auth/user-not-found	No such account	Same message as wrong password
auth/too-many-requests	Firebase rate limit	Too many attempts; try again later
auth/network-request-failed	Offline or blocked	Network error; the page remains usable

Brute-force resistance

Each invalid-credential failure increments `loginAttempts` and the remaining allowance is shown. On the fifth failure `startLoginLockout()` disables both inputs and the button, then runs a one-second interval that counts down thirty seconds before restoring the form and resetting the counter.

Table 7.5: Constants controlling authentication

Constant	Value	Purpose
MAX_LOGIN_ATTEMPTS	5	Failures before the form locks
LOCKOUT_SECONDS	30	Duration of the countdown
Minimum password length	6	Enforced on sign-up

Payment validation

Table 7.6: Per-method payment validation

Method	Fields shown	Validation rule
UPI, Google Pay, PhonePe, Paytm	UPI ID	Must contain an at-sign and be at least five characters
Card	Card number, expiry, CVV	Number at least 12 digits after whitespace is stripped; expiry contains a slash; CVV exactly three digits
Net banking	Bank drop-down	A bank must be selected
(none selected)	-	Prompt to select a method; no state change

7.5 Error Handling

The application is written so that an error degrades one feature without breaking the page. The strategy has four parts: validate early in the browser, wrap every Firebase call in `try/catch`, translate technical errors into readable messages, and guard against asynchronous races.

Table 7.7: Error-handling strategy

Situation	Handling
Invalid form input	Inline message, offending field outlined in red, no network call
Firebase authentication error	`firebaseErrorMessage()` translation; attempts counter on credential failures
Network failure during sign-in	"Network error" message; form stays usable
Database read or write failure	`try/catch`; error logged; history degrades to an empty state rather than a broken page
Wrapper called before Firebase is ready	`waitForFirebase()` waits for the `firebase-ready` event
User leaves the tab during call connection	Live view is written only if the call panel is still displayed

Situation	Handling
Payment input invalid	Rejected with a message; modal stays open
No data yet (prescriptions, history)	Deliberate empty state that tells the user what action produces content
User not signed in when saving history	`saveHistoryEntry()` returns silently
Optional analytics fails to start	Caught and ignored

8. Integration & System Testing

Testing was planned in four layers - unit, black-box, integration and system - and every defect found was recorded in a register. The system was verified by manual testing against the functional requirements, on Chrome, Firefox and Edge at desktop width and at 375 px emulated mobile width.

8.1 Unit Testing

A unit is a single function with no dependence on the page. The pure functions of the application - the triage engine, the escaping function, the medicine lookup, the price lookup and the validators - can be exercised directly from the browser developer console. Table 8.1 lists the unit test cases and their expected results; the same cases can be turned into automated tests with any JavaScript test runner.

Table 8.1: Unit test cases

ID	Unit	Input	Expected result
U-01	getAIResponse	"I have chest pain"	Urgent chest-pain reply
U-02	getAIResponse	"fever and chest pain"	Urgent reply, not the fever reply (priority order)
U-03	getAIResponse	"my ear is blocked"	Fallback asking for onset, severity, other symptoms
U-04	getAIResponse	"FEVER" (upper case)	Same reply as lower case (input lower-cased)
U-05	escapeHtml	"x"	Literal text, no markup interpreted
U-06	getMedsForSpecialty	"Cardiologist"	Aspirin 75 mg and Atorvastatin 20 mg
U-07	getMedsForSpecialty	unrecognised value	General Physician set (fallback)
U-08	computePrescriptionAmount	"Cardiologist"	Rs. 685
U-09	computePrescriptionAmount	unlisted specialty	Rs. 340 (default)
U-10	email check	"abc@"	Rejected
U-11	password check (sign-up)	five characters	Rejected: at least 6 characters
U-12	UPI check	"ab"	Rejected: needs at-sign, at least 5 characters
U-13	card check	eight digits	Rejected: at least 12 digits
U-14	TXN id builder	any call	"TXN" plus last ten digits of Date.now()

```
// run in the browser console on the loaded page
console.assert(getAIResponse("fever and chest pain") === getAIResponse("chest pain"));
console.assert(escapeHtml("<b>x</b>").includes("&lt;"));
console.assert(getMedsForSpecialty("unknown").length > 0);
```

8.2 Black-Box Testing

Black-box testing checks behaviour against requirements without reference to the code. Two standard techniques were used to choose inputs.

Equivalence partitioning

Table 8.2: Equivalence classes

Input	Valid class	Invalid classes
Email	Well-formed address	Empty; missing at-sign; missing domain ("abc@")
Password (sign-up)	Six or more characters	Empty; one to five characters
Confirmation	Equal to password	Different from password
UPI ID	Five or more characters with an at-sign	Shorter than five; no at-sign
Card number	Twelve or more digits	Fewer than twelve digits
CVV	Three digits	Fewer or more than three

Boundary value analysis

Table 8.3: Boundary values

Input	Below boundary	On boundary	Above boundary
Password length	5 -> rejected	6 -> accepted	7 -> accepted
Failed sign-ins	4 -> counter shown	5 -> lockout starts	(locked)
UPI ID length	4 -> rejected	5 with at-sign -> accepted	6 -> accepted
Card digits	11 -> rejected	12 -> accepted	16 -> accepted

Cross-browser and responsive checks

Table 8.4: Browsers and viewports

Environment	Result
Chrome 90+ (desktop)	Full workflow completed
Firefox 88+ (desktop)	Full workflow completed
Edge 90+ (desktop)	Full workflow completed
375 px emulated mobile width	Navigation wraps, doctor grid single column, no horizontal page scroll

8.3 Integration Testing

Integration testing checks that modules work together. An **incremental** approach was used, matching the development increments of Section 3.1: each new module was tested against the ones already in place before the next was added.

Table 8.5: Integration points

Integration	Modules	What is checked
I-A	M1 <-> Firebase Authentication	Sign-up and sign-in succeed through the bridge; errors are translated
I-B	M1 <-> M2	Successful login shows the dashboard; logout returns to login
I-C	M3 <-> UI (both surfaces)	Both chat surfaces use the same engine and keep independent transcripts
I-D	M4 <-> M5	Ending a call with a doctor produces that specialty's prescription
I-E	M5 <-> M7 <-> Firebase	The history record contains the same medicines that were rendered
I-F	M5 <-> M6	The payment amount matches the specialty of the displayed prescription
I-G	M7 <-> M1	Each account sees only its own history

Table 8.6: Integration test cases

ID	Scenario	Expected result
IT-01	Sign up, then log out and log in with the same credentials	Both succeed; dashboard shown each time
IT-02	Consult the cardiologist, end call, open history	Prescription shows Aspirin and Atorvastatin; history lists the same two
IT-03	Consult a dermatologist, then pay	Receipt shows Rs. 420
IT-04	Open the floating widget on the history tab and send a message	Reply appears; page state unchanged
IT-05	Complete a consultation, switch to Home and back	Prescription still displayed in the same session
IT-06	Sign out, sign in as a second account	Only the second account's records are visible

8.4 Test Case Preparation

Test cases were prepared from the functional requirements so that every requirement is covered by at least one case. Test data were disposable accounts and invented symptom sentences; no real patient data were used. Table 8.7 is the master list of system test cases with execution status.

Table 8.7: System test cases

ID	Test case	Expected result	Status
T-01	Submit the login form with both fields empty	Both fields outlined, "Email and password are required"	Pass
T-02	Sign up with a malformed email such as abc@	Email outlined, validity message shown, no network call	Pass
T-03	Sign up with a five-character password	"Password must be at least 6 characters"	Pass
T-04	Sign up with mismatched confirmation	"Passwords do not match"	Pass
T-05	Sign up with an email already registered	Firebase error translated to "account already exists"	Pass
T-06	Sign in with valid credentials	Dashboard shown, doctor grid and history rendered	Pass
T-07	Five consecutive wrong passwords	Form disabled, 30-second countdown, re-enabled afterwards	Pass
T-08	Send "I have chest pain and fever" to the assistant	Urgent cardiology response, not the fever response	Pass
T-09	Send an unmatched message such as "my ear is blocked"	Fallback asking for onset, severity and other symptoms	Pass
T-10	Send HTML markup to the assistant	Rendered as literal text, no markup interpreted	Pass
T-11	Consult the cardiologist and end the call	Prescriptions tab opens with Aspirin and Atorvastatin	Pass
T-12	Consult the dermatologist and end the call	Cetirizine and hydrocortisone issued instead	Pass
T-13	Attempt to edit a medicine on the sheet	No editable control exists; text is not modifiable	Pass
T-14	Press Pay Now with no method chosen	Prompt to select a method; no state change	Pass
T-15	Choose UPI and enter an invalid ID	Rejected as an invalid UPI ID	Pass
T-16	Choose card and enter an eight-digit number	Rejected as invalid card details	Pass
T-17	Complete a valid payment	Processing for 1.8 s, then receipt with amount, method and transaction ID	Pass

ID	Test case	Expected result	Status
T-18	Complete two consultations, then open history	Both listed, most recent first, with correct medicines	Pass
T-19	Sign out, sign in as a second account, open history	Only the second account's records are visible	Pass
T-20	Load the application at 375 px width	Nav wraps, doctor grid single column, no horizontal page scroll	Pass
T-21	Disconnect the network and attempt to sign in	"Network error" message; the page remains usable	Pass

Table 8.8: Requirement-to-test traceability

Requirement	Test cases
FR-01	T-02, T-03, T-04, T-05, IT-01
FR-02	T-06, IT-01
FR-03	T-07
FR-04	T-08, T-09, T-10
FR-05	IT-04
FR-06	T-06
FR-07	T-11, T-12
FR-08	T-11, T-12, IT-02
FR-09	T-13
FR-10	T-17, IT-03
FR-11	T-14, T-15, T-16
FR-12	T-18, IT-02
FR-13	T-19, IT-06
NFR responsiveness	T-20
NFR reliability	T-21

8.5 Bug Tracking

Defects and known issues were kept in a simple register. A defect is *fixed* when a test that failed now passes; a *known issue* is a limitation that was understood and consciously left for future work.

Table 8.9: Bug and known-issue register

ID	Description	Severity	Status
BUG-01	After splitting the single file into four, Firebase functions were not reachable from the classic script	High	Fixed - window bridge and `firebase-ready` event
BUG-02	A fast click on Login could call a Firebase wrapper before the module had loaded	High	Fixed - `waitForFirebase()`
BUG-03	Live view could be written into a hidden panel if the user left the tab while connecting	Medium	Fixed - panel-displayed guard
BUG-04	Chat text containing HTML was interpreted as markup	High	Fixed - `escapeHtml()` on every string
KI-01	Refresh returns to the login screen although the Firebase session is valid	Medium	Open - call `completeLogin()` from the auth observer
KI-02	Lockout counter is in memory and reset by a reload	Low	Open - real limit is Firebase's
KI-03	Only medicine names, not the full sheet, are stored in history	Medium	Open - future scope
KI-04	Patient ID on the prescription is the account email	Medium	Open - use an opaque ID
KI-05	Remote images can disappear	Low	Open - embed images for offline use
KI-06	Icon-only buttons rely on title attributes; chat is not a live region	Low	Open - accessibility work

9. Application Deployment

Deployment makes the application available outside the developer's machine. Because the application is four static files plus a managed backend, any static host will serve it unchanged.

9.1 Cloud Deployment

Table 9.1: Static hosting options

Host	Cost	Notes
Firebase Hosting	Free tier	Same console as Authentication and the database; automatic HTTPS; one-command deploy
GitHub Pages	Free	Deploys straight from the repository; convenient with Section 9.5
Netlify / Vercel	Free tier	Drag-and-drop or Git-based deploys

Deploying with Firebase Hosting

```
npm install -g firebase-tools
firebase login
firebase init hosting      # public directory = the folder holding the four files
firebase deploy
```

After deployment the resulting domain must be added to Authentication, Settings, **Authorized domains**; otherwise sign-in is rejected in production.

9.2 Local Hosting

Because `firebase.js` is an ES module, opening `index.html` directly with a `file://` URL can be blocked by the browser. Serve the folder over HTTP instead.

```
cd smarthealth
python -m http.server 8000
# then open http://localhost:8000
```

Alternatively use the VS Code *Live Server* extension. `localhost` must be in the authorised-domains list.

9.3 APK Build

SmartHealth is a web application and the delivered prototype is not an Android package. If an installable Android app is wanted, the existing web files can be wrapped without rewriting them. Three routes exist: **Capacitor** (wraps the files in a native WebView project), **PWABuilder / Trusted Web Activity** (packages a hosted site that has a web app manifest and a service worker) and a hand-made **Android Studio WebView** project.

```
# Capacitor route (outline)
npm init -y
npm install @capacitor/core @capacitor/cli @capacitor/android
```

```
npx cap init SmartHealth com.example.smarthealth --web-dir=.
npx cap add android
npx cap sync
npx cap open android      # then Build > Build APK(s) in Android Studio
```

Points to check for an APK: the app must be served from an origin Firebase accepts (add it to the authorised domains), remote images require internet access, and the app should be signed with a release key before distribution. These steps are described for completeness; no APK is part of the present deliverable.

9.4 Server Configuration

There is no server of our own to configure. The configuration that matters lives in the Firebase console and in `firebase.json`.

Table 9.2: Configuration checklist

Setting	Where	Required value
Sign-in provider	Authentication, Sign-in method	Email/Password enabled
Authorised domains	Authentication, Settings	localhost and the deployment domain
Realtime Database rules	Realtime Database, Rules	UID-scoped rules of Section 5.5, published
Hosting public directory	firebase.json	Folder with the four files
HTTPS	Hosting	Automatic

```
{
  "hosting": {
    "public": ".",
    "ignore": ["firebase.json", "**/.*", "**/node_modules/**"],
    "headers": [{
      "source": "**/*.@(css|js)",
      "headers": [{ "key": "Cache-Control", "value": "max-age=3600" }]
    }]
  }
}
```

9.5 Version Control Using GitHub

The project source is kept in a Git repository so that every change is recorded, earlier versions can be restored, and the project can be published on GitHub. Figure 9.1 shows the everyday workflow.



Figure 9.1 – Git and GitHub workflow

```
git init
git add index.html style.css firebase.js script.js README.md
git commit -m "Initial commit: SmartHealth AI Platform"
git branch -M main
```

```
git remote add origin https://github.com/<user>/smarthealth.git
git push -u origin main
```

Table 9.3: Repository layout and conventions

Item	Convention
Files	index.html, style.css, firebase.js, script.js, README.md, .gitignore
Branches	`main` is always working; features on short-lived branches (for example `feature/payment-validation`)
Commit messages	Short imperative summary, one logical change per commit
Secrets	The Firebase web config is public by design; service-account keys or admin credentials must never be committed
README	Project summary, set-up steps from Section 9.1-9.2, link to the live site

10. Performance & Security Testing

10.1 Basic Load Testing

SmartHealth has no server of our own, so "load" is shared between the static host, the patient's browser and Firebase. Basic load testing therefore has three questions: how quickly does the page load and respond on an ordinary connection, how much work does the browser do, and what are the limits of the free Firebase plan.

Table 10.1: Performance measures, tools and targets

Measure	Tool	Target
First contentful paint	Chrome Lighthouse (mobile preset)	Under 2.5 s on a typical broadband connection
Total page weight (excluding remote images)	DevTools, Network tab	Small; no framework or build bundle
Interaction responsiveness (tab switch, chat send)	DevTools, Performance tab	Tab switch immediate; chat reply after the designed 700 ms delay
Behaviour on slow network	DevTools throttling (Fast 3G)	Login form and chat usable; images load progressively
History load time with many records	Script that pushes 100 test entries into a test account	Rendered without noticeable delay

Designed delays

Table 10.2: Delays built into the application

Delay	Value	Purpose
AI reply delay	700 ms	Reads as the assistant composing a reply
Call connect delay	1500 ms	Connecting state before the live view
Payment delay	1800 ms	Processing state before the receipt
Lockout duration	30 s	Login lockout countdown

Capacity of the free Firebase plan

The free plan limits the number of simultaneous database connections, stored data and monthly downloads. At the time of writing the free plan allows on the order of one hundred simultaneous connections; current limits should be confirmed on the Firebase pricing page before any demonstration with many users. Each logged-in patient holds roughly one connection, and each history record is a few hundred bytes, so even thousands of consultations occupy a tiny fraction of the storage allowance. A classroom demonstration is therefore well within the limits.

Procedure

1. Open the deployed site in Chrome, run Lighthouse in mobile mode and note the performance score and first contentful paint.

2. Enable network throttling (Fast 3G) and repeat the login and a consultation.
3. Create a test account, push about 100 history entries from the console and open the history tab.
4. Open the site in several browsers and devices at once, signed in as different test accounts, and confirm nothing interferes.
5. Record the observed values in Table 10.3.

Table 10.3: Result sheet (to be filled in during the demonstration)

Measure	Target	Measured value
Lighthouse performance score (mobile)	Record the score	
First contentful paint	Under 2.5 s	
Login on Fast 3G	Usable	
History with 100 entries	No noticeable delay	
Simultaneous test sessions	No interference	

> **Honest reporting**
> The targets above are design goals and test procedures. The measured values are to be recorded by running the procedure on the deployed site, because they depend on the host, the network and the device used.

10.2 Input Validation Checks

Every place where the user can type or choose something was checked for validation, for escaping and for the message shown on rejection.

Table 10.4: Input fields and validation rules

Field	Rule	Rejection behaviour
Login email	Required; must match the email pattern	Field outlined, inline message
Login password	Required	Field outlined, inline message
Sign-up password	At least six characters	"Password must be at least 6 characters"
Confirm password	Must equal password	"Passwords do not match"
Chat message (page and widget)	Escaped before display; empty messages ignored	Markup shown as literal text
UPI ID	Contains an at-sign; at least five characters	Rejected as an invalid UPI ID
Card number	At least twelve digits after whitespace is stripped	Rejected as invalid card details
Card expiry	Contains a slash	Rejected as invalid card details
CVV	Three digits	Rejected as invalid card details

Field	Rule	Rejection behaviour
Bank (net banking)	A bank must be selected	Prompt to select a bank
Payment method	One must be selected	Prompt to select a method

Table 10.5: Malicious input test vectors for the chat

Input	Expected behaviour
`<script>alert(1)</script>`	Displayed as literal text; no alert
`<img src=x onerror=alert(1)>`	Displayed as literal text; no request, no alert
`<b>bold</b>`	Displayed as literal text; not bold
A very long unbroken string	Wraps inside the bubble; layout intact

10.3 Security Validation

Security validation confirms that the countermeasures of Section 5.5 are actually in place. The checklist below maps each control to the way it is verified.

Table 10.6: Security validation checklist

Control	How it is verified	Expected outcome
Passwords never stored by the app	Code review of `handleAuth()`; search script.js for password storage	Password is only passed to the Firebase call
Per-user data isolation	Test T-19; try to read another UID's path from the console while signed in	Second account sees only its own records; foreign path is denied by Firebase
Rules published	Open the Rules tab; use the Rules Playground to simulate an unauthenticated read	Denied
XSS in chat	Test T-10 and the vectors of Table 10.5	No markup interpreted
No account enumeration	Sign in with an unregistered email and with a wrong password	Identical message
Lockout	Test T-07	Form disabled for 30 s
Read-only prescription	Test T-13	No editable control
Authorised domains	Open the site from an unlisted origin	Sign-in rejected
No secrets in the repository	Review the repository for service-account keys	Only the public web config present

Table 10.7: Mapping to common web-application risks (OWASP-style)

Risk category	Status in SmartHealth
Broken access control	Mitigated by UID-scoped rules; depends on rules being published
Injection / cross-site scripting	Mitigated by escaping all displayed text

Risk category	Status in SmartHealth
Identification and authentication failures	Delegated to Firebase; UI lockout is supplementary
Security misconfiguration	Checklist in Section 9.4; authorised domains and rules must be set
Sensitive data exposure	Email printed as patient ID on the sheet - known issue KI-04
Insufficient logging and monitoring	No audit trail - out of scope for the prototype

Residual risks. No consent record, audit trail or practitioner verification exists, and no regulatory compliance (Telemedicine Practice Guidelines, DISHA, HIPAA) is claimed. These are the reasons the system must remain a demonstration.

11. Final Documentation

This final section collects the documents that accompany the finished system: the technical report, the user manual, the application screens and the source code documentation.

11.1 Technical Report

Summary

SmartHealth is a single-page web application that models the full patient journey of a digital healthcare service: account creation, preliminary symptom triage, doctor selection, a video consultation, issuance of a locked electronic prescription, online payment for the prescribed medicines and a persistent record of every past consultation. It is implemented entirely on the client side in four files. All persistence and identity management are delegated to Firebase: Authentication handles email and password sign-up and sign-in, and Realtime Database stores each user's profile and consultation history under a private, UID-scoped path. Triage is performed by a deterministic keyword-matching engine in the browser, and the video consultation and payment are simulated so the workflow can be demonstrated end to end without third-party integrations.

Objectives and outcome

Table 11.1: Objectives achieved

Objective	Outcome
Secure, per-user authenticated entry point	Achieved - Firebase Authentication and UID-scoped rules
Always-available triage assistant	Achieved - rule-based, full page and floating widget
Browse specialists and start a consultation	Achieved - eight-doctor roster, simulated call
Automatic, tamper-resistant e-prescription	Achieved - generated per specialty, read-only
Medicine payment with Indian payment methods	Achieved as a simulation - six methods with validation
Cloud-persisted consultation history	Achieved - newest first, survives logout
Responsive interface for desktop and mobile	Achieved - two breakpoints, tested at 375 px

Technology summary

Table 11.2: Technology stack

Layer	Technology	Role
Structure	HTML5	Semantic markup for all views, modals and forms
Presentation	CSS3	Custom properties, Flexbox, Grid, animation, media queries
Behaviour	JavaScript ES6	Routing, triage engine, prescription generation, payment simulation
Identity	Firebase Authentication	Sign-up, sign-in, sign-out, session observation

Layer	Technology	Role
Persistence	Firebase Realtime Database	User profile and consultation history as a JSON tree
Telemetry	Firebase Analytics	Optional; failure is caught and ignored
Runtime	Modern browser	Chrome 90+, Firefox 88+, Edge 90+, Safari 14+
Editor	VS Code or any text editor	No compiler, bundler or package manager needed

Table 11.3: System requirements

Component	Minimum	Recommended
Processor	Dual-core 1.6 GHz	Quad-core 2.0 GHz or better
Memory	2 GB RAM	4 GB RAM or more
Storage	50 MB free (browser cache)	200 MB free
Display	320 px wide	1366 x 768 or higher
Network	512 Kbps broadband	2 Mbps or better

Limitations

Stating limitations plainly is more useful than omitting them: an examiner will find them, and knowing why each exists is part of the exercise.

Clinical limitations

- **The prescription has no clinician behind it.** Medicines are selected by the doctor's specialty alone. The system never records what the patient said during the call, and the doctor has no input into the sheet. Two patients who consult the same cardiologist for different complaints receive an identical prescription. The system must be treated as a workflow demonstration, never as a source of treatment.
- **Triage is keyword matching, not diagnosis.** A misspelling, an unusual phrasing or a negation such as "I do not have a fever" will not be understood, because the engine only tests for substring presence.

Technical limitations

- **The video call is simulated.** No camera, microphone or WebRTC connection is involved; the panel displays a still image.
- **Payments are simulated.** No gateway is contacted, nothing is charged, and no order record is written to the database.
- **The prescription is not persisted.** Only medicine names reach the history record; reloading returns the prescriptions tab to its empty state.
- **The session is not restored on reload.** The observer updates the current user but does not call `completeLogin()`, so a refresh returns to the login screen.
- **The login lockout is cosmetic.** The counter is in memory; real rate limiting comes from Firebase.

- **There is no doctor role.** Every account is a patient account.
- **All content is remote.** Portraits and backdrops load from Unsplash and Wikimedia; if unreachable the panels render empty.

Security and compliance limitations

- **Configuration is public.** The Firebase config, including the API key, is visible in page source. This is expected for Firebase web apps, but it means database rules and the authorised-domains list are the only real protections.
- **The email address is printed as the patient ID** on the prescription; an opaque patient number would be better practice.
- **No regulatory compliance.** No practitioner registration check, audit trail, consent record, retention policy or compliance with the Telemedicine Practice Guidelines, DISHA or HIPAA.
- **Accessibility is incomplete.** Icon-only buttons rely on title attributes, chat transcripts are not live regions and browser dialogs are used for call feedback.

Future scope

1. **Real video via WebRTC** using `getUserMedia` and a peer connection, with Firebase Realtime Database as the signalling channel.
2. **A doctor portal.** Add a role field to the profile, route doctors to a queue of waiting patients and let the doctor compose the prescription during the call. This removes the most serious clinical limitation.
3. **Model-based triage** with a language model or trained intent classifier that handles paraphrase, negation and misspelling, keeping the current rules as a safety net for red-flag symptoms.
4. **Full prescription persistence and PDF export:** store full medicine objects and re-render any past prescription.
5. **A real payment gateway** such as Razorpay or Stripe with server-side order creation and signature verification.
6. **Session restoration** by calling `completeLogin()` from the auth observer.
7. **Appointment scheduling** with doctor availability and booked slots in the data tree.
8. **Notifications and reminders** using Firebase Cloud Messaging for dose reminders.
9. **Multilingual interface** by extracting all user-facing strings into a language file.

Conclusion of the report

SmartHealth demonstrates that a complete telemedicine workflow - identity, triage, consultation, prescription, payment and record-keeping - can be built from plain HTML, CSS and JavaScript backed by a managed cloud service, with no server of one's own and no build pipeline. The design work is in the joins: triage names a specialist who exists on the platform, ending a call produces the prescription, the prescription offers the payment, and the consultation record is written from the same medicine list that was rendered, so the history can

never contradict the sheet. The prototype's main limitation is also its clearest next step: the prescription comes from a lookup table rather than from the doctor on the call. A doctor-side portal would convert the system from a demonstration of a workflow into a usable implementation of one, and the data model, authentication, history and payment flow are already shaped to receive it.

11.2 User Manual

Before you start

- Use a current version of Chrome, Firefox, Edge or Safari with an internet connection.
- Open the SmartHealth web address supplied by the project team (or `http://localhost:8000` when run locally).
- Remember that SmartHealth is a **demonstration**: do not use its advice or prescriptions for real treatment.

Step-by-step guide

Table 11.4: User manual

Step	What to do	What you will see
1. Create an account	On the login screen choose *Sign up*. Enter your email, a password of at least six characters and the same password again. Press the button.	Your dashboard opens. If a field is wrong, it is outlined in red with a message.
2. Log in	Enter your email and password and press *Login*.	Dashboard. After five wrong attempts the form pauses for 30 seconds.
3. Describe symptoms	Open *AI Assistant* (or the round chat button on any tab). Type how you feel and press Enter or *Send*.	A reply with advice and the specialist to see. Chest pain or breathing trouble gives an urgent warning.
4. Choose a doctor	Open *Telemedicine*. Browse the doctor cards (name, specialty, experience, rating).	A grid of doctors.
5. Start a consultation	Press the start button on a doctor's card.	A short connecting message, then the live consultation view.
6. Use the call controls	Use the microphone and camera buttons if you wish (they are simulated).	A message confirms the action.
7. End the call	Press the red *End Call* button.	The *My Prescriptions* tab opens with your prescription.
8. Read the prescription	Check patient details, medicines, doses, frequency and duration.	A read-only sheet that cannot be edited.
9. Pay for medicines	Press the order button. Choose UPI, Google Pay, PhonePe, Paytm, card or net banking and fill the fields that appear. Press *Pay Now*.	A short processing screen, then a receipt with amount, method and transaction ID.
10. Review history	Open *My History*.	Cards for each consultation, newest first.

Step	What to do	What you will see
11. Log out	Use the sign-out button in the navigation bar.	The login screen.

> **Payment is a simulation**
> No money is charged and no real payment gateway is contacted. The receipt only shows how the real flow would look.

Troubleshooting

Table 11.5: Common problems

Problem	Likely cause	What to do
Nothing happens when opening the file directly	Browser blocks ES modules on `file://`	Open the hosted address, or serve the folder with a local server
Sign-in rejected on the live site	Domain not authorised in Firebase	Add the domain under Authentication, Settings, Authorized domains
"Network error" on login	No internet connection	Reconnect and try again
Form is greyed out after wrong passwords	Five-failure lockout	Wait 30 seconds
Doctor photos missing	Image host unreachable	Check the connection; functionality is unaffected
Prescription tab empty after refresh	The sheet is not stored; only history is	Complete a consultation again; your history is still saved
Returned to login after refresh	Session is not restored in this version	Log in again

11.3 Application Screenshots

This section presents the screens of the application. The figures below are **schematic wireframes** drawn from the interface specification, showing the position and purpose of each element in every screen. For the final printed submission, the wireframes can be replaced with actual screenshots of the running application taken at the same screens, using the table below as the shot list.

Table 11.6: Screen list

Screen	Purpose	Notable elements
Login / sign-up	Entry point for both modes	Gradient over a clinical photograph, inline errors, attempt counter, mode toggle
Home & About	Orientation and feature overview	Hero banner, assistant call-out, feature gallery, mission cards
AI Assistant	Full-page triage conversation	Scrolling transcript, rounded bubbles, Enter-to-send

Screen	Purpose	Notable elements
Telemedicine	Doctor selection and consultation	Responsive doctor grid, video panel, self-view inset, three call controls
My Prescriptions	Display of the issued prescription	Clinical sheet, watermark, medicine table, signature, order button
My History	Record of past consultations	Cards with doctor, specialty, medicines, completion badge, date
Payment modal	Medicine purchase	Six methods, conditional fields, processing state, receipt
Chat widget	Assistant from any tab	Floating button, docked panel, independent transcript

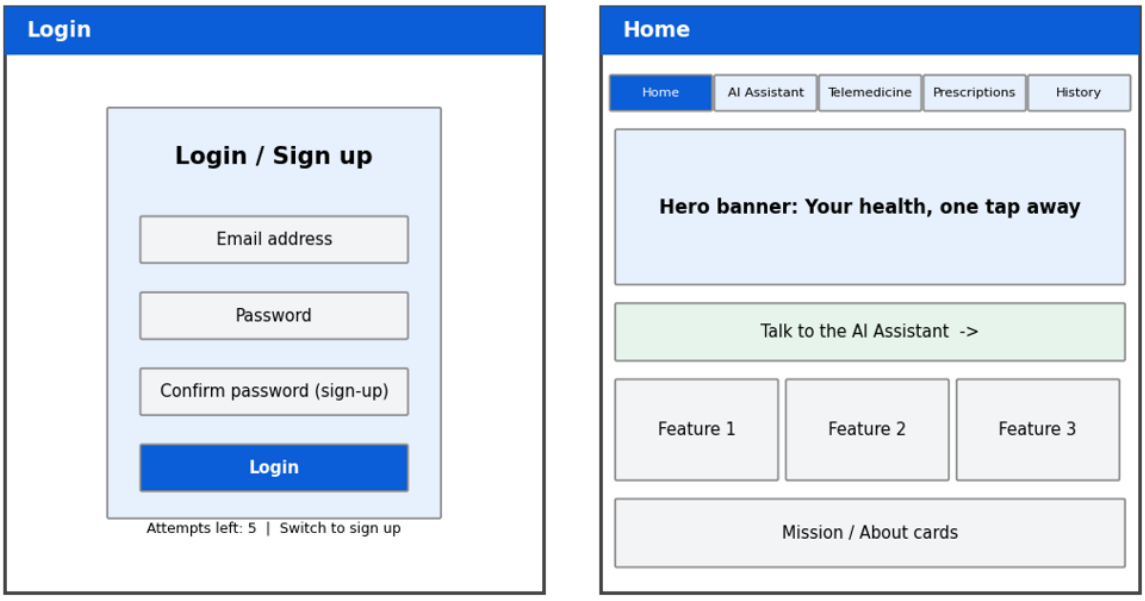


Figure 11.1 – Screens: login / sign-up (left) and home (right) - schematic wireframes

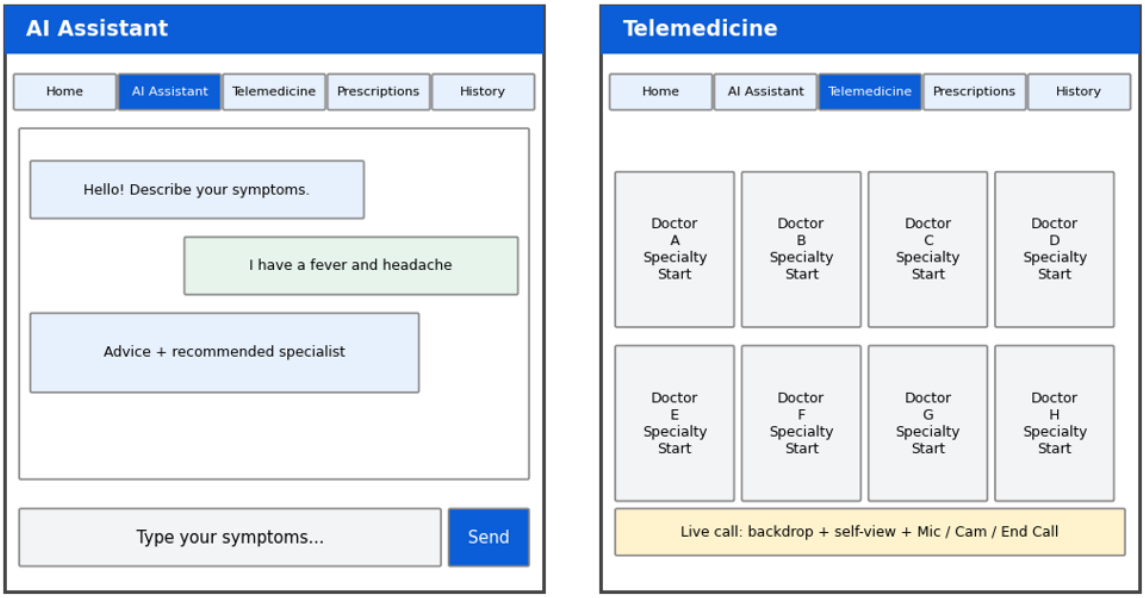


Figure 11.2 – Screens: AI assistant (left) and telemedicine (right) - schematic wireframes

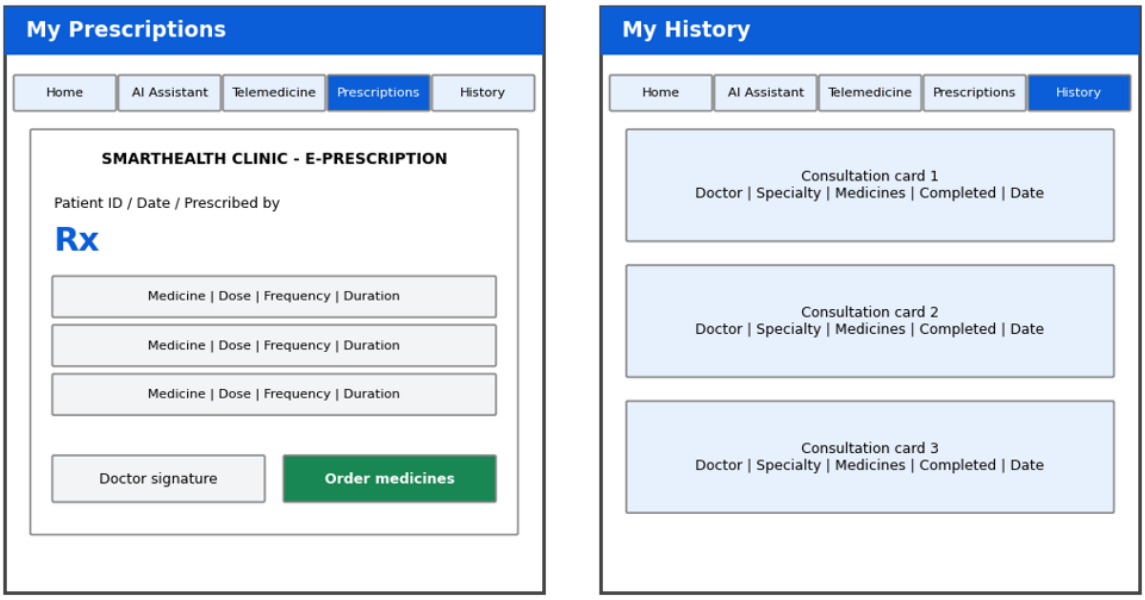


Figure 11.3 – Screens: my prescriptions (left) and my history (right) - schematic wireframes

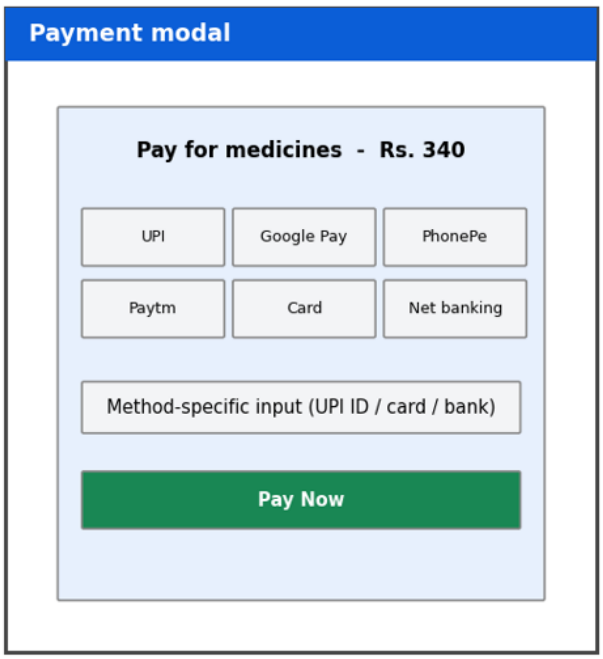


Figure 11.4 – Screen: payment modal - schematic wireframe

11.4 Source Code Documentation

File-by-file description

Table 11.7: Source files

File	Type	Contents
index.html	Markup	Login view, dashboard with five tabs, floating chat widget, payment modal, footer; links style.css, firebase.js and script.js
style.css	Stylesheet	`:root` design tokens, layout (Flexbox and Grid), components, watermark, animations, media queries at 768 px and 420 px
firebase.js	ES module	Imports the Firebase SDK, initialises the app, creates `auth` and `db`, assigns wrapper functions to `window`, sets `firebaseReady` and dispatches `firebase-ready`
script.js	Classic script	Data (DOCTORS, AI_RULES, price map) and logic for modules M1-M7

Function reference

Table 11.8: M1 Authentication and M2 Navigation

Function	Purpose
handleAuth()	Validates the form and performs sign-up or sign-in depending on `authMode`
toggleAuthMode()	Switches between sign-in and sign-up and rewrites labels and visibility
firebaseErrorMessage(code)	Maps a Firebase error code to a readable sentence
startLoginLockout()	Disables the form and runs the 30-second countdown

Function	Purpose
completeLogin()	Hides the login view, shows the dashboard, loads doctors and history
logout()	Signs out and returns to the login screen
switchTab(name)	Shows one view, hides the others, syncs the active nav button; resets the call view or reloads history when needed
waitForFirebase()	Resolves when the Firebase bridge is ready

Table 11.9: M3 AI assistant

Function	Purpose
getAIResponse(message)	Returns the reply of the first rule whose keyword occurs in the lower-cased message, or the fallback
sendTabMessage()	Sends a message from the full-page chat
sendMessage()	Sends a message from the floating widget
escapeHtml(text)	Neutralises markup by round-tripping through `textContent` and `innerHTML`

Table 11.10: M4 Telemedicine, M5 Prescription, M6 Payment and M7 History

Function	Purpose
renderDoctorGrid()	Builds doctor cards from the `DOCTORS` array
startCall(doctor)	Records the doctor, shows connecting state, then the live view after 1.5 s
toggleMic() / toggleCam()	Simulated controls that raise a dialog
endCall()	Clears video, switches to the prescriptions tab, generates the sheet, saves history
getMedsForSpecialty(specialty)	Returns the medicine set for the specialty (default: General Physician)
generatePrescriptionSheet()	Renders the read-only sheet and returns the medicine array
computePrescriptionAmount()	Returns the amount for the specialty (default Rs. 340)
openPaymentModal() / selectPaymentMethod()	Resets the modal and reveals the fields for the chosen method
processPayment()	Validates the revealed fields, shows processing, then the receipt with transaction ID
saveHistoryEntry()	Pushes one consultation record under the user's UID
loadHistory() / renderHistory()	Fetches, sorts descending and renders the history cards or empty state

Configuration constants

Table 11.11: Key constants

Constant	Value and purpose
MAX_LOGIN_ATTEMPTS	5 - invalid-credential failures allowed before the form locks
LOCKOUT_SECONDS	30 - duration of the lockout countdown
AI reply delay	700 ms - pause before the assistant's reply appears
Call connect delay	1500 ms - connecting state before the live view
Payment delay	1800 ms - processing state before the receipt
Default amount	Rs. 340 - charged when a specialty has no entry in the price map
Breakpoints	768 px and 420 px

Coding conventions

- Content (doctors, rules, medicines, prices) is declared as data at the top of `script.js`; logic functions only read it.
- Every Firebase call is wrapped in `try/catch`; user-facing text passes through `escapeHtml()` before insertion.
- Colours, radii and shadows come from CSS custom properties; no colour is hard-coded in a rule outside `:root`.
- Function names are verbs (`renderDoctorGrid`, `startCall`); module state is kept in a small number of named variables.

How to extend the system

Table 11.12: Common extensions

To add...	Do this
A doctor	Add one object (name, specialty, experience, portrait, backdrop) to the `DOCTORS` array
A specialty	Add a branch to `getMedsForSpecialty()` and an entry to the price map
A triage rule	Add an object with keywords and reply to `AI_RULES`, positioned by clinical priority (urgent rules first)
A payment method	Add a button and its fields in index.html and a validation branch in `processPayment()`
A new stored field	Add it to the entry in `saveHistoryEntry()` and read it in `renderHistory()`; no rule change is needed under `users/{uid}`

12. Reference

1. MDN Web Docs. HTML, CSS and JavaScript Documentation. https://developer.mozilla.org/
2. Google Firebase. Authentication Documentation. https://firebase.google.com/docs/auth
3. Google Firebase. Realtime Database and Security Rules Documentation. https://firebase.google.com/docs/database
4. Google Firebase. Hosting Documentation. https://firebase.google.com/docs/hosting
5. WHATWG. HTML Living Standard. https://html.spec.whatwg.org/
6. GitHub. GitHub Documentation. https://docs.github.com/
7. Git. Git Documentation. https://git-scm.com/doc
8. W3C. Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/
9. W3C. WebRTC specification (consulted for future scope). https://www.w3.org/TR/webrtc/
10. OWASP. OWASP Top Ten. https://owasp.org/www-project-top-ten/
11. Ministry of Health and Family Welfare, Government of India. Telemedicine Practice Guidelines, 2020.
12. Object Management Group. Unified Modeling Language Specification. https://www.omg.org/spec/UML
13. Unsplash and Wikimedia Commons. Sources of the photographic assets used in the interface.
14. W3Schools. HTML, CSS and JavaScript Tutorials. https://www.w3schools.com/
15. GeeksforGeeks. Web Development Tutorials and Programming Resources. https://www.geeksforgeeks.org/
16. Pressman, R. S. Software Engineering: A Practitioner's Approach. McGraw-Hill.
17. Sommerville, I. Software Engineering. Pearson.

13. Glossary

Table 13.1: Glossary of terms

Term	Definition
SmartHealth	An AI-assisted telemedicine web application used to triage symptoms, consult a doctor, receive an e-prescription, pay for medicines and keep a consultation history.
Telemedicine	Delivery of medical consultation using electronic communication instead of a physical visit.
Triage	Preliminary assessment of symptoms to decide urgency and which specialist should be consulted.
AI assistant	The rule-based chat component that matches keywords in the patient's message and replies with advice and a recommended specialist.
E-prescription	Electronic prescription generated and displayed by the application after a consultation.
Authentication	Process of verifying a user's identity during login.
Authorization	Process of determining what actions a user is allowed to perform.
Frontend	The visible part of the website through which users interact with SmartHealth.
Backend	The server-side part of an application; in SmartHealth this role is played by Firebase.
BaaS	Backend-as-a-Service: a managed cloud backend such as Firebase.
Firebase	Google platform providing authentication, database and hosting services.
Realtime Database	Firebase cloud database that stores data as a JSON tree.
Security Rules	Server-side rules that decide who can read or write each path of the database.
UID	Unique user identifier assigned by Firebase Authentication.
Push ID	Unique, time-ordered key generated by the Firebase push() operation.
ES module	JavaScript module loaded with import, only from a script of type module.
Bridge	Wrapper functions published on window so that the classic script can use the Firebase module.
API	Application Programming Interface used for communication between software components.
CRUD	Create, Read, Update and Delete operations performed on data.
UML	Unified Modeling Language, used to visually represent the structure and behavior of a system.
Use Case	A description of an interaction between a user and the system to achieve a specific goal.
ER Diagram	Entity Relationship diagram showing entities, attributes and relationships.
SDLC	Software Development Life Cycle, a structured process for developing software.
WBS	Work Breakdown Structure: decomposition of a project into manageable work packages.

Term	Definition
Gantt Chart	Bar chart of project tasks against time.
MoSCoW	Must, Should, Could, Won't: a requirement prioritisation method.
Input Validation	Process of checking whether user-provided data is valid and safe.
XSS	Cross-site scripting: injection of script through unescaped text.
Session	Temporary state that keeps a user logged in.
HTTPS	Secure version of HTTP that encrypts communication between browser and server.
Deployment	Process of making the application available to users on a hosting platform.
GitHub	Platform used for storing, managing and version-controlling source code.
Load Testing	Testing a system under multiple simultaneous users or requests to evaluate performance.
Bug	An error or defect in the software that causes incorrect or unexpected behavior.
Test Case	A defined set of inputs, conditions and expected results used to test a feature.
Responsive Design	Design approach that lets the website work on different screen sizes and devices.
WebRTC	Web technology for real-time audio and video between browsers (future scope).

14. Appendices – SMARTHEALTH

The appendices contain supporting material used during the development, testing and evaluation of the SmartHealth AI Platform.

Appendix A – Survey Questionnaire

A questionnaire can be used to understand the problems that patients face when they need a quick medical opinion, and to evaluate the usefulness of SmartHealth.

Sample Questions:

1. Have you ever avoided or postponed a doctor's visit because of distance, cost or waiting time?
2. Do you search the internet for your symptoms before meeting a doctor?
3. How do you currently get a prescription and keep your medical records?
4. What problems do you face with paper prescriptions or scattered records?
5. Would you use an online consultation platform for minor health problems?
6. Which features would you find most useful (AI assistant, doctor list, video consultation, e-prescription, payment, history)?
7. Is it important that the platform clearly shows when to seek urgent care?
8. Would you be comfortable paying for medicines online?
9. How important are privacy and security of your health information?
10. What improvements would you suggest for SmartHealth?

Appendix B – Survey Data Sheet

The survey data sheet contains the responses collected from users.

Table 14.1: Survey data sheet

Sr. No.	Question	Yes	No	Maybe
1	Have you postponed a doctor's visit?	___	___	___
2	Do you search symptoms online?	___	___	___
3	Would you use SmartHealth?	___	___	___
4	Is a clear urgent-care warning important?	___	___	___
5	Is a stored consultation history useful?	___	___	___

Actual values can be entered according to the survey conducted.

Appendix C – Sample Test Data

Sample records were used to test the SmartHealth application.

Table 14.2: Sample test data

Test ID	Item	Sample value	Purpose
D001	Test account	test1@example.com / 6+ character password	Sign-up and sign-in tests
D002	Test account	test2@example.com / 6+ character password	Data-isolation test (T-19)
D003	Symptom text	"I have chest pain and fever"	Priority-order test (T-08)
D004	Symptom text	"my ear is blocked"	Fallback reply test (T-09)
D005	Doctor	Cardiologist	Aspirin and Atorvastatin issued (T-11)
D006	Doctor	Dermatologist	Cetirizine and hydrocortisone issued (T-12)
D007	UPI ID	user@upi	Valid UPI payment (T-17)
D008	Card number	8 digits	Invalid card test (T-16)

Appendix D – Test Cases

Table 14.3: Summary of test cases

Test Case	Expected Result
Valid sign-in	Patient enters the dashboard
Invalid sign-in	Error message displayed
Five wrong passwords	Form locked for 30 seconds
Chat with the assistant	Relevant advice and specialist shown
Consult a doctor and end the call	Prescription displayed
Pay with a valid method	Receipt displayed
Open My History	Consultations listed, newest first
Sign out and sign in as another user	Only that user's records visible

Appendix E – User Feedback

Users can be asked to provide feedback after using the application. The feedback should focus on:

- Ease of registration and login
- Usefulness of the AI assistant
- Ease of choosing a doctor and completing a consultation
- Clarity of the prescription
- Payment flow
- Consultation history
- User interface and mobile layout